Parametric Divergence Mapping: Deriving Evaluation Criteria for LLM-Assisted Code Artifacts via Alternating Frontier-Model Iteration

Author: Siddharth Nigam

Abstract

Evaluation rubrics for LLM-assisted code artifacts are often written manually, which makes it difficult to tell whether an omitted rule reflects true irrelevance or simply an untested edge case. This paper studies that problem with **Parametric Divergence Mapping (PDM)**, an alternating two-model protocol in which Claude and GPT/Codex edit the same JavaScript measurement script under one narrowly scoped parameter at a time. Each turn must either land one qualifying structural change or return `NULL`; bilateral `NULL`/`NULL` is the convergence signal. The experiment covered four parameters over 77 total iterations: Measurement Accuracy (P1), Edge Case Handling (P2), Coverage Completeness (P3), and Output Clarity (P4) (experiment-log.md). Three parameters converged: P2 at iteration 18, P3 at iteration 14, and P4 at iteration 10; P1 hit its hard cap at 35 without convergence (experiment-log.md, Convergence Record and rows `P3-14`, `P4-10`). The logs also show a repeatable behavioral asymmetry. Claude returned `NULL` 24 times across the full run, whereas GPT/Codex returned `NULL` 3 times, and many Claude `NULL`s were subsequently overturned by Codex structural finds. The experiment produced two concrete artifacts: a production-ready multi-signal measurement script (`P-4/iteration-10.js`, identical to `p4-final.js`) and a tightened v2 methodology built around pre-registered scopes, visible and hidden fixtures, and explicit acceptance rules. The paper's contribution is methodological: PDM does not claim to produce a ground-truth rubric, but it produces a *reproducible* rubric with documented provenance — every entry traceable to the iteration that added it and the iteration that both models declined to modify.

Keywords: LLM benchmarking, cross-model evaluation, machine-readability, agent experience optimization, rubric derivation, methodological convergence.

---

1. Introduction

The motivating application in this repository is Syphon's AXO (Agent Experience Optimization) scoring pipeline, which needed empirically grounded dimensions for measuring how well a page supports agent execution rather than a hand-written checklist (EXPERIMENT_PLAN.md). That application is concrete, but the underlying methodological problem is broader. Any evaluation rubric for LLM-assisted code artifacts, extraction pipelines, or readiness scores faces the same long-tail difficulty: if a scoring dimension is missing, the artifact can still emit clean, stable output while being wrong in exactly the cases a downstream consumer cares about.

The growing practice of LLM-driven web crawling, retrieval-augmented generation, and tool-use agents has renewed interest in a question search engines addressed pragmatically twenty years ago: what does it mean for a web page to be machine-readable? Search-engine optimization (SEO) answered this with a mix of heuristic ranking features and observable crawl behavior, but the answer was never codified as a public scoring rubric. With agent-driven consumption replacing keyword-driven retrieval, the same codification gap reappears for AXO.

Ad hoc human specification is poorly matched to that setting. A researcher can specify obvious dimensions up front, but hand-written rubrics do not reliably expose which distinctions survive contact with actual parser behavior, transport failures, schema consumers, and fixture diversity. The P1 trajectory in this repository demonstrates the failure mode directly: a broad "measurement accuracy" objective admitted a long sequence of still-plausible fixes without any shared stopping rule, and the loop reached 35 iterations without bilateral closure (experiment-log.md, `PARAMETER 1 — ITERATION CAP REACHED`; P-1/iteration-35-changelog.md).

Cross-model iteration is a plausible alternative because disagreement is informative. When two frontier models receive the same active parameter, the same current script, and the same requirement to either land one qualifying change or return `NULL`, several useful signals become observable. Repeated agreement on `NULL` suggests that the remaining move space violates the current acceptance criteria. Repeated disagreement isolates surfaces where the parameter definition is still under-specified or where one model family is systematically stricter than the other. The experiment operationalized that idea by alternating Claude on odd iterations and GPT/Codex on even iterations, storing every artifact and rationale as a changelog in the repository (experiment-log.md; prompts/MASTER_CODEX_PROMPT.md).

The paper's contribution is not an AXO rubric per se. The core contribution is a reproducible procedure for deriving evaluation criteria from structured convergence and divergence on a diffable artifact. AXO is the application case; Parametric Divergence Mapping is the method. This paper does not claim PDM produces a ground-truth rubric; it claims PDM produces a rubric whose entries each have documented provenance, a weaker but falsifiable claim.

---

2. Related Work

PDM sits closest to work on multi-agent debate and consensus, but differs in what is being optimized. Debate-style setups such as Du et al. (2023) use multiple agents or model instances to improve the quality of an answer through contemporaneous exchange. PDM does not ask two agents to jointly solve one problem instance. Instead, it serializes model interaction across repository commits and treats convergence or persistent overturn as evidence about the *rubric* for a code artifact. The closest conceptual overlap is that disagreement is useful rather than purely noisy.

The method also relates to iterative self-improvement work such as Self-Refine (Madaan et al., 2023) and Reflexion (Shinn et al., 2023), both of which improve outputs through repeated critique and revision. The main difference is again the object of optimization and the role of independence. Self-Refine and Reflexion are principally single-model refinement procedures. PDM deliberately alternates model families, stores each turn as a versioned artifact, and treats bilateral `NULL` as the stopping condition. The experiment therefore uses iteration not only to improve a script, but to surface which changes survive adversarially independent re-evaluation.

The paper also intersects with LLM-as-judge work, especially Zheng et al. (2023), which studies when strong models can approximate human preferences, and bias analyses such as Wang et al. (2023), which show that evaluator models themselves introduce systematic distortions. PDM is adjacent but distinct. It does not use one model to score another against a fixed rubric. Instead, it derives the rubric dimensions themselves from iterated model behavior under constrained acceptance rules. This shifts the methodological question from "Can a model judge well?" to "Can disagreement and convergence between models reveal which rubric criteria are stable enough to keep?"

Constitutional AI (Bai et al., 2022) uses a principles list to guide model behavior and iteratively refines those principles. PDM differs in that its principles list (the scope document) is pre-registered and frozen before iteration begins; revision happens only through pre-committed mid-loop review, a mechanism not invoked in this experiment.

Finally, PDM differs from broad benchmark frameworks such as HELM (Liang et al., 2022) and BIG-bench (Srivastava et al., 2022). Those projects evaluate models across many pre-specified tasks, scenarios, and metrics. PDM is narrower and more artifact-centric. It does not benchmark a model family across a task suite. It derives parameter-specific evaluation dimensions for one concrete code artifact whose outputs can be diffed, replayed, and checked against visible and hidden fixtures. In that sense, the work is a rubric-construction methodology rather than a benchmark.

The closest methodological precedent outside the LLM literature is paired-annotator corpus labeling with blind adjudication (e.g., Artstein and Poesio, 2008): two annotators label independently; agreement counts; disagreement is adjudicated. PDM generalizes this from labels to artifact modifications and from human annotators to models. The main difference is that corpus linguistics treats the label space as fixed and disagreement as noise to be resolved; PDM treats scope as fixed but the candidate space as open, and disagreement as either a NULL-breakage signal (one side was wrong) or a boundary signal (the scope does not admit convergence here).

---

3. Method: Parametric Divergence Mapping

3.1 Protocol

PDM uses a simple alternating protocol. A single script is held fixed as the current artifact. One parameter is active at a time. Claude acts on odd-numbered iterations, GPT/Codex on even-numbered iterations, and each turn must do exactly one of two things:

1. land one qualifying structural change under the active parameter, or
2. return `NULL` and copy the current script unchanged.

This structure was present from the start of the experiment and remained stable across parameters (EXPERIMENT_PLAN.md, "NULL Condition Definition"; experiment-log.md, iteration table). The convergence signal is bilateral `NULL`: two consecutive `NULL`s from opposite models on the same script, or an unchanged script accompanied by explicit independent validation that no qualifying move remains. P2 converged in the standard consecutive way at iterations 17-18, while P3 and P4 show the stronger "independent validation on unchanged script" form: Claude returned `NULL`, Codex re-audited the unchanged artifact, and also returned `NULL` (P-2/iteration-17-changelog.md, P-2/iteration-18-changelog.md, P-3/iteration-13-changelog.md, P-3/iteration-14-changelog.md, P-4/iteration-9-changelog.md, P-4/iteration-10-changelog.md).

3.2 Parameter Scoping

The repository shows a clear evolution from loose to pre-registered scoping. P1 ran under the original experiment plan, which described the parameter broadly but did not yet provide a binding include/exclude list or an authoritative fixture suite (EXPERIMENT_PLAN.md, Step 5). By P3 and P4, scope was explicitly pre-registered in standalone documents with INCLUDE and EXCLUDE sections, formal qualification criteria, convergence rules, and hard caps (P3_SCOPE.md; P4_SCOPE.md).

The scoping transition matters methodologically because it prevents relitigation. `P3_SCOPE.md` locks P1 measurement logic and P2 transport logic, bans browser-dependent or constant-valued signals, and requires every new signal to be measurable, meaningful, falsifiable, non-redundant, and additive. `P4_SCOPE.md` narrows the move space even further to schema consistency, type normalization, null-handling explicitness, precision normalization, and rare field renames, explicitly banning additive changes and semantic changes to signals. The result is that later `NULL`s are interpretable as closure under a known search space rather than fatigue under an undefined one.

3.3 Acceptance Criteria

By the end of the experiment, the acceptance gate had been distilled into five concrete rules, stated explicitly in the P4 materials:

1. **Reproducible consumer failure.** A non-`NULL` move must name a concrete consumer access pattern that fails before the change and succeeds after it.
2. **Value preservation.** The move must preserve all field values except the representation change it explicitly introduces.
3. **Cross-path coverage.** The fix must apply across every code path that emits the affected field.
4. **Non-bikeshed motivation.** Renames or schema changes must address a real ambiguity or failure, not stylistic preference.
5. **Idempotence.** Reapplying the move must be equivalent to applying it once.

These criteria are explicit in both the reusable template and the pre-registered scope for P4 (PROMPT_TEMPLATE_P4.md; P4_SCOPE.md). P2 and P3 implement narrower parameter-specific variants of the same idea: P2 admits only crash prevention or fabricated-measurement prevention with red/green fixture evidence, and P3 admits only additive new signals with falsifiability tables and non-redundancy against the existing schema (PROMPT_TEMPLATE.md; PROMPT_TEMPLATE_P3.md).

3.4 Fixture Architecture

The v2 framework separates fixtures into two sets. Visible fixtures are shown to both models and define the immediate falsifiability surface. Hidden holdouts are never shown to either model and are used to detect overfitting (P3_FIXTURES.md; prompts/MASTER_CODEX_PROMPT.md, rule 5). For P3, the researcher also computed authoritative ground-truth values in advance, so proposals could be checked against known signal behavior rather than free-form plausibility (P3_FIXTURES.md).

The architecture contains five visible fixtures and two hidden holdouts. The visible set covers product, news, SPA shell, semantic blog, and div-soup landing-page cases. The hidden set covers local business schema and a hybrid SSR-plus-JS page (P3_FIXTURES.md, "Visible Fixtures" and "Hidden Holdouts"). This design makes two things testable: whether a proposed signal varies at all on the visible set, and whether its apparent novelty survives out-of-distribution examples.

3.5 NULL Semantics

The word `NULL` in this protocol is load-bearing. It is not "I could not think of a change" (which would reward laziness) and not "I do not like any candidate" (which would reward taste). It is a claim that no candidate satisfies all qualification criteria given the current state. A `NULL` is falsifiable: the partner model on the next iteration either produces a qualifying change (overturning the `NULL`) or independently returns `NULL` (ratifying it). The partner's response is the adjudicator.

This falsifiability makes `NULL` a *finding*, not an absence. When Claude returned `NULL` at P4 iteration 5 (P-4/iteration-5-changelog.md) and Codex overturned it at iteration 6 (P-4/iteration-6-changelog.md), Claude's `NULL` was *wrong* — and in this case the wrongness was diagnosed specifically: Claude had used JavaScript `typeof` to audit type drift, which collapses integer and float into the same runtime type, while the downstream failure mode lived at the JSON-bytes level, which Codex inspected directly. When Claude returned `NULL` at iteration 9 and Codex returned `NULL` at iteration 10, both models ratified the same state. The two outcomes are epistemically distinct, and the log preserves the distinction. Section 7.4 analyzes the methodological consequence: once a `NULL` is broken, the next `NULL` must explicitly name the failure mode it missed and probe that new surface directly.

3.6 v1 → v2 Evolution

The repository does not contain a single standalone five-point postmortem for P1, but the v2 redesign is directly reconstructible from the protocol artifacts.

Design pressure exposed by P1	v2 correction in the repo	Evidence
Broad parameter definition without a mechanical stopping rule	P2 narrows qualification to crash prevention or fabrication prevention and disqualifies wording-only changes	PROMPT_TEMPLATE.md
No authoritative falsifiability surface	P3 requires visible-fixture diversity tables and precomputed ground truth	PROMPT_TEMPLATE_P3.md, P3_FIXTURES.md
Scope relitigation across iterations	P3 and P4 introduce binding scope documents with INCLUDE/EXCLUDE lists and hard caps	P3_SCOPE.md, P4_SCOPE.md
No explicit overfit check	P3 adds hidden holdouts and requires post-change holdout comparison	P3_SCOPE.md, P3_FIXTURES.md
Conversational state burden across turns	The master Codex prompt externalizes iteration state to repo files and instructs the model to infer the active parameter and iteration from the repository rather than from conversational memory	prompts/MASTER_CODEX_PROMPT.md

The progression from P2 to P4 shows a clear tightening trend: reproducible red/green fixtures in P2, scope plus visible/hidden fixture discipline in P3, and explicit consumer-failure pairs plus value-preservation rules in P4. PDM therefore matured during the experiment. The later parameters are not merely additional trials; they are trials under a substantially better-controlled protocol.

---

4. Experimental Setup

4.1 Task

The artifact under refinement was a Node.js script that began as a DOM-depth measurer and ended as a multi-signal AXO measurement tool. The baseline `original-script.js` fetched a URL server-side and emitted only `maximum_depth`, `average_depth`, `element_count`, `timestamp`, and `error` (EXPERIMENT_PLAN.md, Step 3; original-script.js). Through P3 and P4, the script became a 17-key output object with eleven nested signal groups plus the original depth fields (P-4/iteration-10.js).

4.2 Models

Claude acted on odd iterations and GPT/Codex on even iterations throughout the experiment (experiment-log.md, "Experiment Design"). Later protocol materials standardized the Codex side further. `prompts/MASTER_CODEX_PROMPT.md` instructs Codex to determine the current state from repository files, forbids reading hidden holdouts, and treats the repository rather than the conversation as the source of truth for iteration state (prompts/MASTER_CODEX_PROMPT.md).

4.3 Fixtures

The P3/P4 fixture suite contained 5 visible fixtures and 2 hidden holdouts. Ground-truth values for the visible suite were researcher-computed and documented in `P3_FIXTURES.md` before P3 iteration 1 (P3_FIXTURES.md). The visible set was intentionally constructed to vary on the seeded structural axes, while the hidden set tested whether the accepted signals generalized beyond the fixture patterns the models could see.

4.4 Per-parameter Caps and Scope

Parameter	Goal	Cap	Scope controls	Final artifact
P1 Measurement Accuracy	browser-faithful DOM depth measurement	35	original v1 framing only	P-1/iteration-35.js
P2 Edge Case Handling	clean errors instead of crashes or fabricated measurements	20	v2 prompt with mandatory reproducible fixture rule	P-2/iteration-18.js
P3 Coverage Completeness	additive new AXO-relevant signals	20	pre-registered scope, visible/hidden fixtures, ground truth	P-3/iteration-14.js
P4 Output Clarity	schema consumability without value changes	10	pre-registered scope, move categories A-E, consumer-failure pairs	P-4/iteration-10.js

The P3 and P4 caps were justified explicitly in the scope documents, which cite P1's scope drift as the motivating failure case for tighter control (P3_SCOPE.md; P4_SCOPE.md).

---

5. Results

5.1 P1 Measurement Accuracy

P1 did not converge. The canonical convergence record reports 35 iterations, 14 Claude `NULL`s, 0 GPT `NULL`s, 4 Claude structural changes, and 17 GPT structural changes (experiment-log.md, `PARAMETER 1 — ITERATION CAP REACHED`). The important result is not only that the cap was reached, but *how* it was reached. GPT/Codex repeatedly found browser-equivalence defects after Claude had declared closure: template opacity at iteration 4, implicit table wrappers at iteration 6, full document-wrapper synthesis at iteration 8, and `parseNoneClosedTags` at iteration 32 (experiment-log.md, "Parametric Divergence Observations"; P-1/iteration-4-changelog.md, P-1/iteration-6-changelog.md, P-1/iteration-8-changelog.md, P-1/iteration-32-changelog.md).

The same convergence record groups P1 fixes into parser configuration, document synthesis, table wrapper synthesis, traversal transparency, and traversal opacity, and leaves three unresolved deferrals: foster parenting, adoption-agency behavior, and the select content model (experiment-log.md, `PARAMETER 1 — ITERATION`

`CAP REACHED`). Those deferrals all require HTML5 tree-construction state unavailable from the parser output, which means P1 was effectively trying to solve browser-faithful DOM reconstruction without an oracle, without fixtures, and without a tight boundary around what counts as "done."

5.2 P2 Edge Case Handling

P2 converged because the failure surface collapsed around one observable bad output: the fabricated synthesized three-element document, reported as `maximum_depth: 1`, `average_depth: 0.7`, `element_count: 3`, `error: null` on inputs that should not have been measured (experiment-log.md, `PARAMETER 2 — BILATERAL CONVERGENCE ACHIEVED`; P-2/iteration-15-changelog.md; P-2/iteration-16-changelog.md).

The convergence path is best understood as closure of five causal axes of the same symptom:

Failure axis	Representative iterations	Evidence
Transport / HTTP handling	P2-1, P2-2, P2-4, P2-8	stack overflow prevention, no-content rejection, partial-content rejection, catch-chain repair (P-2/iteration-1-changelog.md, P-2/iteration-8-changelog.md)
Auto-parse under claimed HTML	P2-10, P2-11, P2-12	falsy coercion fix and upstream non-string rejection (P-2/iteration-10-changelog.md, P-2/iteration-11-changelog.md, P-2/iteration-12-changelog.md)
Declared charset mismatch	P2-14	UTF-16LE mojibake rejected upstream (P-2/iteration-14-changelog.md)
Pre-parse body shape	P2-15	empty, whitespace-only, and plain-text bodies under `text/html` rejected (P-2/iteration-15-changelog.md)
Post-parse zero-element markup	P2-16	`<`, comment-only, CDATA, and similar zero-element bodies rejected with doctype exemption (P-2/iteration-16-changelog.md)

The last two iterations are especially informative. Claude's iteration 17 `NULL` argues that all five axes are now closed and classifies remaining candidates as not-a-fabrication, speculative, or wording-only (P-2/iteration-17-changelog.md). Codex iteration 18 then independently rechecks doctype shells, empty HTML shells, SVG/XML with real elements, malformed-doctype junk, BOM-prefixed HTML, and transport failures, and also returns `NULL` (P-2/iteration-18-changelog.md). This is a clean example of bilateral closure under a tightened fixture-based rule.

5.3 P3 Coverage Completeness

P3 converged at iteration 14 after accepting 11 new signal groups: `js_dependency`, `structured_data`, `heading_hierarchy`, `semantic_html`, `machine_readable_blocks`, `answer_first`, `link_profile`, `selector_affordance`, `content_depth`, `text_container_profile`, and `reference_context` (experiment-log.md, rows `P3–1` through `P3–14`; Appendix D). Each signal was required to add a new schema field, vary across the visible fixtures, and clear a non-redundancy test against existing signals (PROMPT_TEMPLATE_P3.md; P3_SCOPE.md).

The important empirical pattern is not just that 11 signals were accepted, but that redundancy pressure became increasingly visible. Six accepted signals carry explicit weak-boolean-redundancy caveats in the fixture notes: `semantic_html`, `machine_readable_blocks`, `link_profile`, `selector_affordance`, `content_depth`, and `text_container_profile` (P3_FIXTURES.md, sections "Signal Values After P3 Iteration 4", "5", "7", "8", "9", and "10"). In several of those cases, the signal survived only because sub-fields or holdout behavior rescued a boolean projection that would otherwise have collapsed to an earlier pattern. `reference_context` went further still: it was accepted for its link-location subfield decomposition even

though Claude later documented that its derived `mode` category was fully recoverable from prior signals on all seven fixtures (P-3/iteration-12-changelog.md; P3_FIXTURES.md, "Signal Values After P3 Iteration 12").

P3 closes with a strong example of independent validation. Claude iteration 13 rejects `temporal_metadata` as semantically real but non-redundant only in theory, not on the current fixture set (P-3/iteration-13-changelog.md). Codex iteration 14 then also returns `NULL`, explicitly stating that "Claude's iter-13 analysis was correct" and rejecting `temporal_metadata`, `section_heading_coverage`, and `article_header_profile` under the same non-redundancy criterion (P-3/iteration-14-changelog.md). The experiment log records this as bilateral convergence at 14 with the coverage-completeness space saturated under the current fixture suite (experiment-log.md, row `P3-14`).

5.4 P4 Output Clarity

P4 is the cleanest illustration of the mature method. It produced six non-`NULL` moves and four `NULL` attempts, with bilateral convergence reached at the hard cap of 10 iterations (experiment-log.md, rows `P4-1` through `P4-10`).

Iteration	Model	Outcome	Category / issue
P4-1	Claude	non-`NULL`	category A: catch-block key-set drift (P-4/iteration-1-changelog.md)
P4-2	GPT/Codex	non-`NULL`	category C/B: `heading_hierarchy` null-object (P-4/iteration-2-changelog.md)
P4-3	Claude	non-`NULL`	category D: `average_depth` precision normalization (P-4/iteration-3-changelog.md)
P4-4	GPT/Codex	non-`NULL`	category B: all remaining nested groups object-always (P-4/iteration-4-changelog.md)
P4-5	Claude	`NULL`	later broken by iter-6 (P-4/iteration-5-changelog.md)
P4-6	GPT/Codex	non-`NULL`	category B: Python-visible int/float normalization via fixed-width float serializer (P-4/iteration-6-changelog.md)
P4-7	Claude	`NULL`	later broken by iter-8 (P-4/iteration-7-changelog.md)
P4-8	GPT/Codex	non-`NULL`	category B: serializer collision on sentinel-shaped `url` input (P-4/iteration-8-changelog.md)
P4-9	Claude	`NULL`	defensible `NULL` after explicit adversarial audit (P-4/iteration-9-changelog.md)
P4-10	GPT/Codex	`NULL`	bilateral convergence (P-4/iteration-10-changelog.md)

Two methodological failure modes are documented in the changelogs themselves. First, Claude's iteration-5 `NULL` used JavaScript runtime typing and therefore missed the fact that Python `json.loads` distinguishes `2` from `2.00`; Codex iteration 6 broke that `NULL` with a real consumer failure on `weighted_avg_depth` and `hook_density` (P-4/iteration-5-changelog.md, P-4/iteration-6-changelog.md). Second, Claude's iteration-7 `NULL` corrected the audit surface to byte-level JSON inspection but still limited the audit to fixture-path inputs; Codex iteration 8 then showed that the serializer itself could corrupt the top-level `url` field on adversarial sentinel-shaped input (P-4/iteration-7-changelog.md, P-4/iteration-8-changelog.md). By iteration 10, both models agreed that the remaining space had collapsed from ambiguity to preference. Codex's convergence rationale states: "What remains is not ambiguity in the emitted JSON. It is preference. That is exactly where P4 is supposed to stop." (P-4/iteration-10-changelog.md).

---

6. Behavioral Findings

6.1 Claude/Codex Asymmetry

The strongest behavioral result is the `NULL` asymmetry. Counting directly from the canonical iteration table yields 24 Claude `NULL`s and 3 GPT/Codex `NULL`s across the four parameters (experiment-log.md, iteration table). Broken down by parameter:

Parameter	Claude `NULL`s	GPT/Codex `NULL`s
P1 Measurement Accuracy	14	0
P2 Edge Case Handling	5	1
P3 Coverage Completeness	2	1
P4 Output Clarity	3	1

The logs show a consistent directional difference in behavior. Claude tends to ask whether the remaining move is truly within scope and sufficiently structural to justify continuation. Codex tends to ask whether any definitional or consumer-visible defect still survives. The repository supports this as behavior, not mechanism: P1's convergence record explicitly summarizes Claude as optimizing "scope control and practical impact thresholds" and GPT as optimizing "definitional correctness regardless of implementation cost" (experiment-log.md, `PARAMETER 1 — ITERATION CAP REACHED`). P4's broken `NULL`s show the same asymmetry under a much tighter parameter: Claude stopped once the pre-registered issue list looked closed, whereas Codex continued to search for consumer-visible type corruption and serializer edge cases (P-4/iteration-5-changelog.md, P-4/iteration-6-changelog.md, P-4/iteration-8-changelog.md).

6.2 The Claude-NULL / Codex-Structural-Find Pattern

This pattern recurs across parameters:

Parameter	Claude `NULL`	Codex overturn
P1	iter 3	iter 4 finds template opacity (experiment-log.md, "P-1: Iteration 3-4 Divergence")
P1	iter 5	iter 6 finds implicit table wrappers (experiment-log.md, "P-1: Iteration 5-6 Divergence")
P1	iter 7	iter 8 synthesizes HTML/HEAD/BODY wrappers (experiment-log.md, "P-1: Iteration 7-8 Divergence")
P1	iter 31	iter 32 enables `parseNoneClosedTags` (experiment-log.md, "P-1: Iteration 31-32 Divergence")
P2	iter 5	iter 6 improves descriptive HTTP status text within the parameter definition (experiment-log.md, "P-2: Iteration 5-6 Divergence")
P2	iter 7	iter 8 fixes catch-branch ordering bug (experiment-log.md, "P-2: Iteration 7-8 Divergence")
P2	iter 13	iter 14 finds charset-mismatch fabrication (experiment-log.md, "P-2: Iteration 13-14 Divergence")
P4	iter 5	iter 6 finds Python-visible int/float drift (P-4/iteration-6-changelog.md)
P4	iter 7	iter 8 finds serializer collision on adversarial input (P-4/iteration-8-changelog.md)

The pattern does not imply that Claude was simply "too conservative" in every case. P2 iteration 6 is a good counterexample: the change did not alter success-versus-failure control flow, but it still fell within the stated parameter definition because P2 still allowed "descriptive message" quality at that point (P-2/iteration-6-

changelog.md; PROMPT_TEMPLATE.md, change log from v1). The important empirical point is narrower: Codex was more willing to keep searching after Claude judged the surface saturated.

6.3 Convergent Follow-Through

Not all cross-model interaction took the form of overturning a prior `NULL`. Several of the most productive sequences involved the second model extending the exact failure class just exposed by the first.

In P2, Codex iteration 10 found falsy-coercion fabrication on headerless bodies, and Claude iteration 11 generalized that same auto-parse failure class to claimed-HTML responses (P-2/iteration-10-changelog.md, P-2/iteration-11-changelog.md; experiment-log.md, "P-2: Iteration 10-11 Follow-through"). Codex iteration 14 then found the encoding axis of the fabricated-three-element bug, and Claude iteration 15 extended the same class to empty, whitespace-only, and plain-text `text/html` bodies (P-2/iteration-14-changelog.md, P-2/iteration-15-changelog.md; experiment-log.md, "P-2: Iteration 14-15 Follow-through"). Codex iteration 16 extended that one layer deeper again, from pre-parse body shape to post-parse zero-element markup (P-2/iteration-16-changelog.md; experiment-log.md, "P-2: Iteration 15-16 Follow-through").

At convergence, the same pattern appeared in a stricter form. Claude iteration 13 in P3 returned `NULL` with a detailed rejection of `temporal_metadata`. Codex iteration 14 did not merely fail to find a new move; it explicitly validated the same rejection and adopted the same non-redundancy conclusion (P-3/iteration-13-changelog.md, P-3/iteration-14-changelog.md).

Independent arrival at the same `NULL` across models is the protocol's strongest convergence signal. It is epistemically distinct from a ratification where the second model simply returns `NULL`: in independent arrival, the two models' reasoning chains, constructed without access to each other, converge on the same rejection criterion. Two of the three bilateral convergences in this experiment (P3 iteration 14, P4 iteration 10) closed this way.

6.4 Meta-Methodology Emergent from P4

P4 produced the clearest methodological lesson in the repository: once a `NULL` is broken, the next `NULL` has to name the failure mode it missed and then audit that surface directly.

Claude's iteration-7 changelog makes this explicit. It begins by correcting iteration 5's methodology, stating that the earlier `NULL` used the wrong surface because JavaScript `typeof` collapses integer and float values into the same runtime type (P-4/iteration-7-changelog.md). Claude's iteration-9 changelog repeats the pattern at a higher bar: it opens by distinguishing itself from iterations 5 and 7 and then documents eight concrete audit surfaces, including adversarial serializer probes that the earlier `NULL`s had not run (P-4/iteration-9-changelog.md).

This is not a generic moral; it is an empirical protocol refinement recorded in the artifacts. P4 shows that `NULL` quality itself must be audited. A `NULL` is methodologically credible only if it probes the surface that broke the last `NULL`.

6.5 NULL That Does Not Hold

P4 iterations 5 and 7 are the clearest examples of methodologically flawed `NULL`s.

Iteration 5 concluded that no live P4 move remained because it diffed success-path leaf runtime types in JavaScript and found no drift (P-4/iteration-5-changelog.md). Codex iteration 6 showed that this missed the relevant consumer surface entirely: Python consumers treat whole-number JSON tokens as `int`, so designated float fields still drifted across fixtures (P-4/iteration-6-changelog.md).

Iteration 7 corrected the type-audit surface but still dismissed serializer fragility as hypothetical refactoring rather than a live consumer failure (P-4/iteration-7-changelog.md). Codex iteration 8 then converted that fragility into a reproducing counterexample by passing `__AXO_FLOAT__2.00__` as the URL and showing that the output corrupted `"url"` from string to float (P-4/iteration-8-changelog.md). The lesson is straightforward: a `NULL` that only inspects the fixture surface can still fail when the parameter is defined over downstream consumability rather than fixture values.

6.6 Model-dependence of findings

A question PDM's methodology raises but cannot settle on the evidence of one model pair: are the rubric entries that emerge from PDM *model-dependent*? If the experiment were re-run with GPT-4 in place of Codex, or Gemini in place of Claude, would the convergence points change?

On the evidence of this experiment, the answer is plausibly yes at the margin and plausibly no at the core. The six weak-boolean-redundancy signals in P3 are the marginal cases: each was accepted on a decomposition argument that a different model pair might have rejected. The P2 failure-class axes are the core cases: both models independently recognized the fabricated-three-element pattern and closed it consistently across transport, encoding, and body-shape axes. The core is more likely to replicate; the margin less so. This is a falsifiable prediction and motivates the cross-pair replication listed under future work (§9.1).

---

7. What PDM Produces and Does Not Produce

7.1 What PDM produces

PDM produces a rubric whose entries each have a documented provenance: the iteration that added the entry, the iterations that declined to modify or remove it, and the rejection reasons for nearby candidates that did not make the cut. Every entry answers the question "why is this here, and not something else?" with a specific log reference.

This is a weaker claim than "PDM produces a correct rubric." No mechanism in PDM validates the rubric against external ground truth. The rubric is correct *relative to the scope document*, which the researcher wrote. A badly-scoped parameter produces a badly-grounded rubric (P1 is the experiment's own negative example). PDM shifts accountability from "the researcher chose these signals" to "the researcher chose this scope, and these signals are what two models independently converge on under it" — still a researcher-driven artifact, but with the signal-choice step externalized.

7.2 What PDM does not produce

PDM does not produce signal weights. Coverage Completeness (P3) converged on eleven signals; the experiment does not claim those eleven signals are equally important for scoring a page's machine-readability. Weight derivation requires a different instrument: either labeled data (pages rated as agent-usable or agent-hostile) or a downstream pipeline whose success is measurable (task-completion rates). This paper addresses only criterion derivation, not weighting.

PDM does not produce per-signal thresholds. P3 added `js_dependency.dependent` as a boolean signal, but "what value of `script_count` should flip the boolean" was set by the proposing model's internal judgment, not by convergence. A principled threshold derivation would run a second-order PDM loop on the threshold function parameters, which this paper does not attempt.

PDM does not produce generalization guarantees. The eleven signals came from seven fixtures. Fixtures were representative by construction but not statistical. A signal converged-on in PDM may fail on an unseen fixture class (the weak-boolean-redundancy flags are explicit acknowledgments of this risk).

---

8. Discussion

8.1 What Bilateral NULL/NULL Actually Signals

Bilateral `NULL` in PDM is not a claim of absolute truth. It is a claim that, under the current parameter definition, current acceptance criteria, and current fixture architecture, two independently prompted frontier models both failed to identify a qualifying move. P2-18 says this directly: remaining candidates were "not crashes, not fabrications, already clean errors, or speculative defensive checks" (P-2/iteration-18-changelog.md). P4-10 states the same idea even more sharply: "What remains is not ambiguity in the emitted JSON. It is preference." (P-4/iteration-10-changelog.md). In other words, bilateral `NULL` marks procedural closure, not metaphysical completeness.

8.2 Why P1 Did Not Converge

P1 failed for reasons that later protocol documents address directly, even if the repository does not package them as a single five-item postmortem.

First, P1 lacked an authoritative ground-truth oracle. The later P3 fixture suite explicitly says that ground-truth values had to be computed before iteration 1 because otherwise models "cannot be held to the falsifiability rule" (P3_SCOPE.md, final paragraph; P3_FIXTURES.md).

Second, P1's notion of a qualifying structural move was too vague. The original plan defined `NULL` generically as the absence of a structural improvement, but later v2 materials had to narrow that substantially. `PROMPT_TEMPLATE.md` explicitly records that P2 dropped a "correct error classification" clause because it was undefined and invited cosmetic or message-quality moves (PROMPT_TEMPLATE.md).

Third, P1's scope was too broad. Both `P3_SCOPE.md` and `P4_SCOPE.md` cite P1 scope drift to 35 iterations without convergence as the reason for binding INCLUDE/EXCLUDE lists and hard caps (P3_SCOPE.md; P4_SCOPE.md).

Fourth, P1 had no falsifiability discipline. P2 adds mandatory red/green fixture evidence, P3 requires visible-fixture diversity tables, and P4 requires a concrete consumer-failure pair. None of that structure existed for P1 (PROMPT_TEMPLATE.md; PROMPT_TEMPLATE_P3.md; PROMPT_TEMPLATE_P4.md).

Fifth, the later protocol also reduced the burden of accumulated conversational state by externalizing iteration context into repository files and a reusable master prompt (prompts/MASTER_CODEX_PROMPT.md). The repository does not isolate "accumulated context bias" as a separately quantified causal factor, but it does show that later iterations were deliberately redesigned to depend less on free-form conversation state and more on repo-state reconstruction.

Taken together, P1 was an under-instrumented search for browser-faithful DOM equivalence. P2-P4 were controlled experiments.

8.3 Generalizability

PDM is not a general substitute for human evaluation of open-ended generation. It depends on a specific kind of task: the artifact must be diffable, its outputs must be replayable, and at least part of the failure surface must

be observable via fixtures, downstream consumer accesses, or both. That is why the method fits a measurement script well and would fit other deterministic or semi-deterministic code-quality tools, but would be much less convincing for unconstrained creative generation.

8.4 Cost

The direct cost is easy to count: 35 + 18 + 14 + 10 = 77 iterations, each of which required one model inference plus human operator work to save artifacts, run checks, and update logs (experiment-log.md; EXPERIMENT_PLAN.md). The repository does not track token counts, wall-clock latencies, or operator minutes separately, so the paper cannot report more granular cost numbers. The method is therefore inexpensive relative to large benchmark construction, but not free: it trades broad annotation expense for iterative operator supervision on one artifact.

8.5 No external validity check

Nothing in PDM validates that pages scoring well under the derived rubric are, in fact, more machine-readable than pages scoring poorly. This paper does not perform a downstream pipeline evaluation. The AXO framework is planned to run exactly this evaluation in a subsequent phase ("Part 2" of the project, which integrates the P4-final script into Syphon's scoring pipeline and measures agent task-completion rates against pages rated by the rubric), but that phase is outside this paper's scope. Bilateral `NULL` proves scope-internal stability; it does not prove external utility. Any claim PDM makes about rubric quality is contingent on the eventual Part 2 evaluation.

---

9. Limitations

- The experiment used two model families, not a larger jury. The observed asymmetry may not replicate with different pairings.
- The repository documents one researcher operating the loop. No inter-rater reliability measurement exists.
- Iteration caps were pragmatic choices justified in scope documents, not statistically optimized stopping rules (P3_SCOPE.md; P4_SCOPE.md).
- The category system for P4 moves (A-E) was introduced only when the output-clarity parameter began, not from the start of the full experiment (PROMPT_TEMPLATE_P4.md; P4_SCOPE.md).
- The dominant P2 bug class, the fabricated synthesized three-element document, is specific to this script and parser stack. The methodology generalizes better than the bug class does.
- The repository does not include a standalone P1 postmortem file quantifying every proposed root cause. Some discussion in Section 8.2 therefore reconstructs protocol lessons from later documents rather than quoting a single canonical postmortem.
- Seven fixtures is a small sample. Weak-boolean-redundancy flags in P3 are explicit acknowledgments that adding fixtures could force re-classification. A production rubric derivation would run PDM on a fixture set of 20 or more, with holdouts proportionally larger.
- Full reproduction requires running two LLM services at specific versions, neither of which offers retrospective version access. The protocol is reproducible; the specific convergence points may not be.

---

10. Future Work

- **Cross-model-pair replication.** Replicate the protocol with different model pairings (GPT-4 + Gemini, Llama + Mistral) on the same scope documents and fixtures. Convergence points that replicate across pairs are strong candidates for model-independent rubric entries. Convergence points that differ reveal model-pair-dependent artifacts, which is itself useful information.
- **Formalize the divergence signal itself.** A model's overturn-rate against prior `NULL`s may be a useful calibration statistic.
- **Three-model PDM.** Add a third-model adjudication step for disputed `NULL`s so that broken versus defensible `NULL` can be classified automatically. Expected to tighten convergence (fewer signals accepted) but make `NULL` breakage rarer.
- **Weight derivation via downstream evaluation.** The eleven signals from P3 need weights before they form a scoring function. Weight derivation via PDM would require a parameter like "P5 — Signal Weighting," with moves of the form "increase weight of `structured_data` relative to `selector_affordance` by factor X" and a convergence condition tied to a held-out task-completion evaluation.
- **Apply PDM to other rubric-derivation tasks.** PDM is not specific to machine-readability scoring. Any domain where a researcher would like to derive a multi-criterion rubric from LLM consensus — code-quality rubrics, academic-writing rubrics, design-review rubrics — can in principle run the same protocol on different artifacts.
- **Automate mid-loop review.** Instrument the log with convergence-rate metrics (NULLs-per-iteration trend, fixture output stability, signal-count growth rate) and flag the review point programmatically.
- **Downstream integration.** Integrate `p4-final.js` as the production AXO measurement script and test whether the derived dimensions improve end-to-end scoring behavior in Syphon (EXPERIMENT_PLAN.md, Part 2).

---

11. Conclusion

The repository provides a complete worked example of deriving evaluation criteria rather than assuming them. Across 77 alternating iterations, PDM produced one non-converged parameter, three converged parameters, a production-ready measurement script, and a visibly tighter experimental framework than the one it started with. The central empirical result is not that two frontier models eventually agreed. It is that their pattern of disagreement and closure exposed which rubric dimensions were still unstable, which ones survived independent re-audit, and which methodological safeguards were necessary before a `NULL` could be trusted. For deterministic code artifacts with replayable outputs, that is a useful contribution in its own right.

PDM does not produce ground-truth rubrics; it produces reproducible rubrics with documented provenance. That distinction is the methodology's honest contribution and its primary limitation.

---

12. Appendix A — Full Iteration Log Table

A.1 Parameter 1

```
| Iteration | File | Agent | Parameter | Changes Made | NULL |
|-----------|------|-------|-----------|--------------|------|
| 1 | P-1/iteration-1.js | Claude | Measurement Accuracy | Fixed phantom root node
miscounting when `<html>` tag absent — virtual root wrapper (tagName null) was counted
as element | NO |
| 2 | P-1/iteration-2.js | GPT | Measurement Accuracy | Replaced querySelector('html')
with direct-child search to avoid anchoring on nested `<html>` in malformed HTML | NO |
| 3 | P-1/iteration-3.js | Claude | Measurement Accuracy | No change — tested implicit
elements, SVG, void elements, rounding, consistency, deep nesting | YES |
| 4 | P-1/iteration-4.js | GPT | Measurement Accuracy | Stop traversal at `<template>`
elements — parser exposes template contents as children but browser DOM puts them in
DocumentFragment | NO |
| 5 | P-1/iteration-5.js | Claude | Measurement Accuracy | No change — tested iframe
fallback, noscript, script/style, multiple html elements, DOCTYPE | YES |
| 6 | P-1/iteration-6.js | GPT | Measurement Accuracy | Synthesize implicit table
wrapper elements (TBODY, TR, COLGROUP) that browsers insert but node-html-parser omits |
NO |
| 7 | P-1/iteration-7.js | Claude | Measurement Accuracy | No change — considered
implicit head/body insertion (real but disproportionate fix), li/p/option auto-closing
(parser handles correctly) | YES |
| 8 | P-1/iteration-8.js | GPT | Measurement Accuracy | Synthesized browser document
wrappers (HTML/HEAD/BODY), routed elements by head-vs-body classification, made nested
HTML/HEAD/BODY transparent in traversal | NO |
| 9 | P-1/iteration-9.js | Claude | Measurement Accuracy | No change — considered foster
parenting, nested forms, adoption agency algorithm, select content model. All require
HTML5 tree construction state not available from parser | YES |
| 10 | P-1/iteration-10.js | GPT | Measurement Accuracy | Added frameset document
handling — no synthetic BODY for frameset docs, FRAMESET at HTML-child depth, BODY under
NOFRAMES preserved, FRAME registered as void tag | NO |
| 11 | P-1/iteration-11.js | Claude | Measurement Accuracy | No change — considered
deprecated void elements (keygen/command/menuitem), table caption, explicit colgroup,
math namespace. None affect measurement on modern HTML | YES |
| 12 | P-1/iteration-12.js | GPT | Measurement Accuracy | Added keygen, basefont,
bgsound, command to voidTagNames — browser treats as void elements but parser was
dropping them | NO |
| 13 | P-1/iteration-13.js | Claude | Measurement Accuracy | No change — verified void
element list is complete, checked SVG self-closing, ruby annotations, remaining tree
construction mismatches | YES |
| 14 | P-1/iteration-14.js | GPT | Measurement Accuracy | Added blockTextElements config
for title, textarea, xmp — browser treats content as raw text, parser was creating
phantom child elements | NO |
| 15 | P-1/iteration-15.js | Claude | Measurement Accuracy | Enabled fixNestedATags
parser option — browsers close outer `<a>` on inner `<a>`, parser was nesting them | NO
|
| 16 | P-1/iteration-16.js | GPT | Measurement Accuracy | Enabled closeAllByClosing
parser option — mismatched closing tags were dropping open ancestor elements | NO |
| 17 | P-1/iteration-17.js | Claude | Measurement Accuracy | No change — exhaustively
audited all parser Options fields from TypeScript definitions. Every tree-structure-
affecting option is either enabled or correctly excluded | YES |
| 18 | P-1/iteration-18.js | GPT | Measurement Accuracy | Removed `pre` from
blockTextElements — browsers parse child elements inside `<pre>`, it's not a raw-text
element like script/textarea | NO |
| 19 | P-1/iteration-19.js | Claude | Measurement Accuracy | Removed `noscript` from
blockTextElements — browsers parse child elements inside body noscript, suppressing them
is a structural undercount | NO |
```

```
| 20 | P-1/iteration-20.js | GPT | Measurement Accuracy | Stop traversal inside head-
context noscript — browsers treat head noscript as raw text with scripting enabled,
completing the context split from iter 19 | NO |
| 21 | P-1/iteration-21.js | Claude | Measurement Accuracy | No change — audited
remaining blockTextElements, head-context noscript edge case (documented limitation),
void tag completeness, plaintext element | YES |
| 22 | P-1/iteration-22.js | GPT | Measurement Accuracy | Fixed head-context noscript in
document synthesis — stop traversal at routing site when no explicit HEAD tag,
completing iter 21's documented limitation | NO |
| 23 | P-1/iteration-23.js | Claude | Measurement Accuracy | No change — verified all
fixes, no unfixed issues identified | YES |
| 24 | P-1/iteration-24.js | GPT | Measurement Accuracy | Extended head-context opacity
from NOSCRIPT to NOFRAMES — same pattern, browsers don't expose child elements in head-
context NOFRAMES | NO |
| 25 | P-1/iteration-25.js | Claude | Measurement Accuracy | No change — systematically
audited all 11 headTagNames entries, every one now handled via void tags,
blockTextElements, or traversal stops | YES |
| 26 | P-1/iteration-26.js | GPT | Measurement Accuracy | Removed BODY-under-NOFRAMES
preservation — Chrome doesn't expose nested BODY under NOFRAMES, BODY now always
transparent | NO |
| 27 | P-1/iteration-27.js | Claude | Measurement Accuracy | No change — verified BODY
transparency correctness in all contexts, no remaining special cases | YES |
| 28 | P-1/iteration-28.js | GPT | Measurement Accuracy | Made NOFRAMES fully opaque in
traversal (not just head-context) — Chrome doesn't expose NOFRAMES children in any
context | NO |
| 29 | P-1/iteration-29.js | Claude | Measurement Accuracy | No change — verified
NOSCRIPT vs NOFRAMES distinction is correct (NOSCRIPT body children ARE real DOM nodes,
NOFRAMES children are not) | YES |
| 30 | P-1/iteration-30.js | GPT | Measurement Accuracy | Made nested FORM elements
transparent — Chrome drops inner form but keeps descendants, traversal ancestor check
instead of parse-time state | NO |
| 31 | P-1/iteration-31.js | Claude | Measurement Accuracy | No change — remaining items
(foster parenting, adoption agency, select content model) all require reparenting, not
transparency | YES |
| 32 | P-1/iteration-32.js | GPT | Measurement Accuracy | Enabled `parseNoneClosedTags`
parser option — EOF-truncated documents were losing unclosed elements that browsers
implicitly close | NO |
| 33 | P-1/iteration-33.js | Claude | Measurement Accuracy | No change — verified no
negative interactions with parseNoneClosedTags, preserveTagNesting correctly excluded,
no new traversal fixes possible | YES |
| 34 | P-1/iteration-34.js | GPT | Measurement Accuracy | Added `iframe` and `noembed`
to blockTextElements — browsers don't parse child elements inside these, parser was
creating phantom children | NO |
| 35 | P-1/iteration-35.js | Claude | Measurement Accuracy | Added `plaintext` to
blockTextElements — same raw-text pattern as iframe/noembed, browsers don't parse
children inside plaintext | NO |
```

A.2 Parameter 2

```
| Iteration | File | Agent | Parameter | Changes Made | NULL |
|-----------|------|-------|-----------|--------------|------|
| P2-1 | P-2/iteration-1.js | Claude | Edge Case Handling | Converted recursive
traversal to iterative (prevents stack overflow on deep nesting), added Content-Type
validation, added response size limit | NO |
| P2-2 | P-2/iteration-2.js | GPT | Edge Case Handling | Added HTTP 204/205 no-content
handling — returns clean error instead of fabricated HTML/HEAD/BODY measurement | NO |
| P2-3 | P-2/iteration-3.js | Claude | Edge Case Handling | Non-string response.data
guard (coerce to string), content sniffing for missing Content-Type (reject non-HTML
bodies without headers) | NO |
| P2-4 | P-2/iteration-4.js | GPT | Edge Case Handling | Added HTTP 206 Partial Content
rejection — partial byte-range responses produce unreliable DOM measurements | NO |
| P2-5 | P-2/iteration-5.js | Claude | Edge Case Handling | No change — verified all
crash/silent-failure paths covered (invalid URLs, parser resilience, empty bodies,
circular redirects, 3xx without Location) | YES |
| P2-6 | P-2/iteration-6.js | GPT | Edge Case Handling | Use `err.response.statusText`
in HTTP error messages (e.g. `HTTP 304: Not Modified` in place of generic `Server
returned an error`); generic fallback preserved when statusText absent | NO |
| P2-7 | P-2/iteration-7.js | Claude | Edge Case Handling | No change — iter 6 accepted
as in-definition; remaining message-quality tweaks are cosmetic;
protocol/encoding/2xx/sniff edge cases either handled or pathological | YES |
| P2-8 | P-2/iteration-8.js | GPT | Edge Case Handling | Fixed catch-branch ordering —
axios tags HTTP 5xx with `ERR_BAD_RESPONSE` and that branch over-caught before
`err.response`, misreporting every 5xx as "Response exceeded maximum size (50 MB)";
moved `err.response` first and narrowed `ERR_BAD_RESPONSE` special case to messages
mentioning `maxContentLength` | NO |
| P2-9 | P-2/iteration-9.js | Claude | Edge Case Handling | No change — iter 8 closed
the last catch-chain misclassification; remaining candidates
(ERR_FR_TOO_MANY_REDIRECTS/EAI_AGAIN/ECONNRESET branches, scheme pre-rejection, argv
trimming, ERR_BAD_RESPONSE detection via `!err.response`) are message polishing or
regressions in disguise | YES |
| P2-10 | P-2/iteration-10.js | GPT | Edge Case Handling | Removed `|| ''` in non-string
response coercion — falsy parsed values (`0`, `false`, `null`) were being collapsed to
empty string, then measured as 3-element synthesized HTML instead of rejected by the
missing-Content-Type sniff | NO |
| P2-11 | P-2/iteration-11.js | Claude | Edge Case Handling | Added upstream reject for
auto-parsed non-string `response.data` — axios's default `transformResponse` auto-parses
JSON, so a non-string body is a structural signal the content is not HTML. Closes the
silent-failure path for servers that send JSON under `Content-Type: text/html` (sniff
was skipped, parser fabricated a 3-element measurement) | NO |
| P2-12 | P-2/iteration-12.js | GPT | Edge Case Handling | Dropped `null` from iter 11's
exclusion list — verified axios returns `""` for empty bodies, not `null`, so the `null`
exclusion was overfit and let literal JSON `null` bodies slip through as 3-element
fabricated measurements | NO |
| P2-13 | P-2/iteration-13.js | Claude | Edge Case Handling | No change — iter 12 closed
the auto-parse silent-failure class; remaining candidates (dropping the `undefined`
exclusion, removing the now-inert `String()` coercion) are cosmetic dead-code removal
under current axios behavior, not structural edge-case handling | YES |
| P2-14 | P-2/iteration-14.js | GPT | Edge Case Handling | Added declared-charset
validation — reject responses whose `Content-Type` declares a charset other than utf-
8/utf8/us-ascii/ascii. Axios decodes all text responses as UTF-8 regardless of declared
charset, so a valid UTF-16LE HTML body arrived as mojibake and measured as a 3-element
synthesized document. Reproduced on `charset=utf-16le` fixture | NO |
| P2-15 | P-2/iteration-15.js | Claude | Edge Case Handling | Hoisted body-shape sniff
out of the `!contentType` guard and made it run unconditionally; added explicit
empty/whitespace-only body rejection. Under `Content-Type: text/html` the missing-
Content-Type sniff was skipped, so empty, whitespace-only, and tags-free bodies all
reached the parser and produced the fabricated 3-element synthesized-document
```

```
measurement. Same silent-failure class as iter 10/11/12 and iter 14, on the body-shape
axis | NO |
| P2-16 | P-2/iteration-16.js | GPT | Edge Case Handling | Added post-parse zero-element
guard with doctype exemption — catches bodies that pass iter 15's pre-parse `<` sniff
but produce no element nodes after parsing (`<` alone, `<!-- comment only -->`, `<?xml?
>`, CDATA, `</` trailing slash). Fifth axis of the fabricated 3-element class: the pre-
parse sniff accepts "starts with `<`" but element-free markup bodies still fall through
to the HTML/HEAD/BODY synthesis path. Doctype-only bodies exempted to preserve existing
measurement behavior | NO |
| P2-17 | P-2/iteration-17.js | Claude | Edge Case Handling | No change — all five axes
of the fabricated 3-element class are closed (transport, auto-parse, encoding, body-
shape pre-parse, zero-element post-parse). Remaining candidates are (a) contested iter
16 policy choices (doctype exemption), (b) not-a-fabrication cases (`<html></html>`,
SVG-only, XML with real elements), (c) speculative defensive checks (doctype preceded by
garbage), or (d) v2 disqualifiers (error-message wording, dead-code cleanup, redirect-
loop branch specificity). Crash paths all resolve to clean error JSON | YES |
| P2-18 | P-2/iteration-18.js | GPT | Edge Case Handling | No change — rechecked doctype
shells, empty HTML shells, SVG/XML with real elements, malformed-doctype junk, BOM-
prefixed HTML, and transport failures. None produced a qualifying crash or fabricated
measurement under the tightened v2 fixture rule | YES |
```

A.3 Parameter 3

```
| Iteration | File | Agent | Parameter | Changes Made | NULL |
|-----------|------|-------|-----------|--------------|------|
| P3-1 | P-3/iteration-1.js | Claude | Coverage Completeness | Added `js_dependency`
nested field group (`script_count`, `body_text_length`, derived `dependent` boolean).
Distinguishes JS-required SPA shells from server-rendered content for no-browser agent
scraping. Falsifiable: 2 distinct `dependent` values across 5 visible fixtures (true on
V3 SPA, false on V1/V2/V4/V5). H1/H2 holdouts both correctly `false` — H2 (SSR summary +
2 script bundles) is the threshold case that would false-positive under a script-count-
only rule | NO |
| P3-2 | P-3/iteration-2.js | GPT | Coverage Completeness | Added `structured_data`
nested field group (`json_ld_script_count`, `microdata_node_count`, `rdfa_node_count`,
`schema_types` array, derived `present` boolean). Supports fact-extraction agent tasks
(product/business identity) without brittle DOM scraping. Falsifiable: `present=true` on
V1 only across visibles (schema_types: AggregateRating, Brand, Offer, Product). Overfit
check PASSES — H1 holdout correctly flips to `present=true` (schema_types:
LocalBusiness, PostalAddress), H2 correctly `false`. Signal is not flat on holdouts | NO
|
| P3-3 | P-3/iteration-3.js | Claude | Coverage Completeness | Added `heading_hierarchy`
nested field group (`h1_count`, `heading_count`, `skipped_levels`, derived `valid` =
`h1_count==1 AND heading_count>=2 AND skipped_levels==0`). Supports page-outline /
summarization agent tasks — heading-based extraction requires a clean hierarchy.
Falsifiable: `valid` true on V1/V2/V4, false on V3/V5 (no headings). Non-redundant:
distinguishes V2 (SSR news, valid) from V5 (SSR divsoup, invalid) — prior signals could
not. Holdouts correctly both `valid=true` (H1 h1+3h2, H2 h1+h2); sub-field
`heading_count` differs (4 vs 2) so signal is not flat on holdouts | NO |
| P3-4 | P-3/iteration-4.js | GPT | Coverage Completeness | Added `semantic_html` nested
field group (`semantic_element_count`, `landmark_count`, `tags_present` array, derived
`present` boolean on count>0). Supports main-content isolation / chrome suppression for
extraction agents. Falsifiable: `present` splits V1/V2/V4 (true) vs V3/V5 (false);
`semantic_element_count` has 4 distinct values (0, 8, 9, 11). **Non-redundancy note:**
derived `present` happens to coincide with `heading_hierarchy.valid` on every visible
fixture — a weak redundancy on the boolean. Sub-fields are genuinely non-redundant: V1
(8) vs V4 (11) were indistinguishable across all prior signals (both
false/false/true/true). Holdouts: H1=8/4/present, H2=6/4/present; `present` flat on
holdouts but sub-fields vary | NO |
| P3-5 | P-3/iteration-5.js | Claude | Coverage Completeness | Added
`machine_readable_blocks` nested field group (`table_count`, `time_count`,
`address_count`, `dl_count`, `figure_count`, `total`, derived `present` boolean on
total>0). Supports in-content micro-data extraction — specs tables, machine-parsable
dates, contact addresses, definition lists, captioned figures. Falsifiable: `present`
true on V1/V2/V4, false on V3/V5; `total` has 3 distinct values (0, 1, 2); `table_count`
isolates V1 (1) from V2/V4 (0). **Non-redundancy note:** derived `present` matches
`heading_hierarchy.valid` on every visible — same weak-boolean-redundancy pattern as
iter 4. Saved by orthogonality on holdout H2 (heading_hierarchy.valid=true,
semantic_html.present=true, **machine_readable_blocks.present=false**) — first signal
where H2 breaks from the prior-signal majority. H1 exercises `address_count=1` which is
flat at 0 on all visibles, so sub-fields measure real variation that visibles don't
expose | NO |
| P3-6 | P-3/iteration-6.js | GPT | Coverage Completeness | Added `answer_first` nested
field group (`top_third_signal_count`, `bottom_third_signal_count`, `signal_delta`,
derived `front_loaded` = top>=bot AND top>0). Claimed task: fast first-pass answering —
measures whether informative content density is front-loaded vs back-loaded.
Falsifiable: `front_loaded` true on V1/V4, false on V2/V3/V5; top/bot counts have 3
distinct values each. Non-redundancy: distinguishes V2 (false) from V4 (true) — they
shared every prior boolean. Holdouts: H1=true (1/1), H2=false (0/1). **Methodological
concern flagged (Claude, not Codex):** the `informativeSentencePattern` regex mixes
generic patterns with fixture-specific content words, creating partial fixture leakage |
NO |
```

P3-7	P-3/iteration-7.js	Claude	Coverage Completeness	Added `link_profile` nested field group (`link_count`, `navigational_link_count`, `fragment_link_count`, `contact_link_count`, derived `navigable` = nav_count>0, derived `fragment_only` = link>0 AND nav_count==0). Supports crawl-out decisioning, fake-nav detection, and direct-contact actionability. Falsifiable: 3 distinct `link_count` values on visibles (0, 3, 5); `fragment_only` isolates V5 (T) — V5 has 5 fake-anchor `#` links while V3 has 0 links, a structural pattern no prior signal distinguishes. Non-redundancy: V3 vs V5 were identical on 5 of 6 prior signals (only `js_dependency` split them); `fragment_only` adds a second orthogonal distinguishing axis. Holdouts: H1 exercises `contact_count=2` (mailto+tel, flat at 0 on visibles); H2=navigable:T, different from its F pattern on mrb/answer_first	NO
P3-8	P-3/iteration-8.js	GPT	Coverage Completeness	Added `selector_affordance` nested field group (`id_count`, `classed_element_count`, `data_attribute_element_count`, `hooked_element_count`, `hook_density`, derived `selector_rich`). Task: pre-render extraction-strategy choice — selector-first automation vs semantic fallback. Falsifiable: 3 distinct `hooked_element_count` values and 5 distinct `hook_density` values. **Non-redundancy — honest note:** on visibles the derived boolean `selector_rich` is fully predictable as `structured_data.present OR js_dependency.dependent OR link_profile.fragment_only`. Saved by holdouts: H1 and H2 break the prior-signal prediction in opposite directions. Sub-fields carry further non-redundant information	NO
P3-9	P-3/iteration-9.js	Claude	Coverage Completeness	Added `content_depth` nested field group (`text_node_count`, `substantive_text_node_count`, `weighted_avg_depth`, `deepest_substantive_depth`, derived `prose_heavy`). Task: readability/summarization routing — whether a page carries real prose under chrome versus being a chrome-heavy wrapper. Falsifiable on every numeric sub-field. **Non-redundancy — key claim:** `prose_heavy` gives a boolean pattern no prior signal had on visibles. **Weak-boolean-redundancy on visibles:** `prose_heavy=T` coincides with `body_text_length>=500`, but holdouts flip that proxy in both directions. Text-node-level measurement is the genuinely new axis	NO
P3-10	P-3/iteration-10.js	GPT	Coverage Completeness	Added `text_container_profile` nested field group (`paragraph_text_node_count`, `list_text_node_count`, `tabular_text_node_count`, `presentational_text_node_count`, `heading_text_node_count`, `other_text_node_count`, derived `dominant_block_type`). Task: choose extraction primitive before chunking. Falsifiable: 4 distinct `dominant_block_type` values on visibles. **Weak-boolean-redundancy check:** collapsing `dominant_block_type==paragraph` yields the same visible pattern as `content_depth.prose_heavy`, but the categorical signal and sub-counts remain non-redundant. H1 produces `dominant=mixed`, a categorical value no visible exercises	NO
P3-11	P-3/iteration-11.js	Claude	Coverage Completeness	No change — scanned remaining structural axes and found only flat, thin, or boolean-redundant candidates. First Claude NULL of P3	YES
P3-12	P-3/iteration-12.js	GPT	Coverage Completeness	Added `reference_context` nested field group (`chrome_navigable_link_count`, `contextual_navigable_link_count`, `aside_navigable_link_count`, `non_chrome_fragment_link_count`, derived `mode`). Task: decide whether a page contributes useful follow-on references or only site chrome. Falsifiable: 4 distinct `mode` values on visibles. **Non-redundancy concern:** Claude later shows `mode` is fully derivable from prior signals on all 7 fixtures; the counts remain non-redundant. Accepted on sub-field decomposition grounds	NO
P3-13	P-3/iteration-13.js	Claude	Coverage Completeness	No change — strongest candidate `temporal_metadata` rejected as semantically real but redundant on the current fixture set. Second Claude NULL of P3	YES
P3-14	P-3/iteration-14.js	GPT	Coverage Completeness	No change — bilateral convergence achieved. Codex rejects `section_heading_coverage`, `temporal_metadata`, and `article_header_profile` under the same non-redundancy rule and explicitly validates Claude's iter-13 temporal-metadata rejection	YES

A.4 Parameter 4

```
| Iteration | File | Agent | Parameter | Changes Made | NULL |
|-----------|------|-------|-----------|--------------|------|
| P4-1 | P-4/iteration-1.js | Claude | Output Clarity | Category A (schema-consistency
fix). Added `heading_hierarchy: null` and `machine_readable_blocks: null` to the top-
level `catch (err)` block so all 11 code paths emit the same 17 top-level keys. Consumer
failure before: `json.loads(out)["heading_hierarchy"]` raised `KeyError` on catch-path
outputs | NO |
| P4-2 | P-4/iteration-2.js | GPT | Output Clarity | Category C/B. Converted
`heading_hierarchy` on all non-success paths from bare `null` to an explicit null-object
so consumers can access `heading_hierarchy.valid` without `TypeError` | NO |
| P4-3 | P-4/iteration-3.js | Claude | Output Clarity | Category D. Normalized
`average_depth` from 1-decimal to 2-decimal precision to align with the other float-
designated fields | NO |
| P4-4 | P-4/iteration-4.js | GPT | Output Clarity | Category B. Applied the object-
always pattern to all 10 remaining nested signal groups via `createNullSignalGroups()`
so non-success paths preserve leaf-key stability | NO |
| P4-5 | P-4/iteration-5.js | Claude | Output Clarity | No change — NULL. Declared the
pre-registered P4 issue list closed after iter 1-4 and found no remaining leaf-type
drift using a JavaScript-runtime audit | YES |
| P4-6 | P-4/iteration-6.js | GPT | Output Clarity | Category B. Added a fixed-width
float serializer so Python `json.loads` sees the three designated float fields as
`float` even when their numeric value is a whole number (`2.00`, `0.00`) | NO |
| P4-7 | P-4/iteration-7.js | Claude | Output Clarity | No change — NULL. Corrected
iter-5's audit to byte-level JSON inspection but still dismissed serializer fragility as
hypothetical rather than a live consumer failure | YES |
| P4-8 | P-4/iteration-8.js | GPT | Output Clarity | Category B. Replaced the sentinel-
based serializer with key-anchored rewrites after demonstrating that iter-6/7 could
corrupt the top-level `url` field on sentinel-shaped adversarial input | NO |
| P4-9 | P-4/iteration-9.js | Claude | Output Clarity | No change — NULL. Re-audited
fixture outputs plus six adversarial serializer inputs, explicitly addressing the two
prior broken NULL failure modes. Defensible NULL | YES |
| P4-10 | P-4/iteration-10.js | GPT | Output Clarity | No change — bilateral convergence
at the hard cap. Codex re-audited the post-iter-8 surface and concluded that remaining
differences were preference, not ambiguity | YES |
```

13. Appendix B — Fixture Specifications and Ground-Truth Values

B.1 Fixture Inventory

```
## Visible Fixtures (shown to models)

| #  | File                            | Category                                    |
Purpose
|
|----|---------------------------------|---------------------------------------------|----
-----------------------------------------------------------------------------------------
-------------|
| V1 | `visible/v1-product.html`       | E-commerce product page                     |
JSON-LD Product + Offer + AggregateRating; spec table; one JS-LD script; semantic
structure             |
| V2 | `visible/v2-news.html`          | News article, clean heading hierarchy       |
Article wrapper; one H1, multiple H2s with H3s nested; time element; no JavaScript
|
| V3 | `visible/v3-spa.html`           | JavaScript-dependent SPA shell              |
Near-empty body (`<div id="root"></div>` + noscript); multiple script tags; no real
content in raw HTML |
| V4 | `visible/v4-blog.html`          | Semantic-rich blog post                     |
Uses header/nav/main/article/section/aside/footer; no JSON-LD; strong structure signals
|
| V5 | `visible/v5-divsoup.html`       | Div-only marketing landing page             | No
semantic tags; no structured data; no headings; no JavaScript; just nested divs carrying
text        |

## Hidden Holdouts (NEVER shown to models)

| #  | File                            | Category                                    |
Purpose
|
|----|---------------------------------|---------------------------------------------|----
-----------------------------------------------------------------------------------------
-------------|
| H1 | `holdout/h1-localbiz.html`      | Local business with LocalBusiness schema    |
Overfit check for structured-data + entity signals
|
| H2 | `holdout/h2-hybrid.html`        | Hybrid — summary SSR, interactive JS        |
Overfit check for JS-dependency threshold (has scripts AND real content)
|
```

B.2 Baseline Outputs Before P3

```
| #  | Fixture          | maximum_depth | average_depth | element_count |
|----|------------------|---------------|---------------|---------------|
| V1 | v1-product       | 9             | 4.9           | 48            |
| V2 | v2-news          | 6             | 4.0           | 39            |
| V3 | v3-spa           | 2             | 1.7           | 13            |
| V4 | v4-blog          | 7             | 4.0           | 38            |
| V5 | v5-divsoup       | 8             | 4.7           | 46            |
| H1 | h1-localbiz      | 6             | 3.9           | 35            |
| H2 | h2-hybrid        | 5             | 2.9           | 25            |
```

B.3 Ground-Truth Summary Across Accepted P3 Signals

```
| #  | Fixture          | js_dep (iter 1) | struct_data (iter 2) | head_hier (iter 3) |
sem_html.present (iter 4) | mrb.present (iter 5) | answer_first (iter 6) | link_profile
(iter 7) | selector_affordance (iter 8) | content_depth (iter 9) |
text_container_profile (iter 10) | reference_context (iter 12) |
|----|------------------|-----------------|----------------------|--------------------|-
--------------------------|----------------------|----------------------|-------------
---------|------------------------------|----------------------|---------------------
-------------|------------------------------|
| V1 | v1-product       | false           | **true**             | **true**           |
**true** (sem=8)          | **true** (total=2)   | **front_loaded**      | **navigable**
(3)     | **selector_rich** (hooked=1, density=0.02) | not prose_heavy (sub=2,
wavg=4.92) | **tabular** (tab=7, para=6) | **chrome_only** (chrome=3) |
| V2 | v2-news          | false           | false                | **true**           |
**true** (sem=9)          | **true** (total=1)   | not front_loaded      | **navigable**
(3)     | not selector_rich (hooked=1, density=0.03) | **prose_heavy** (sub=7,
wavg=4.81) | **paragraph** (para=9) | **chrome_only** (chrome=3) |
| V3 | v3-spa           | **true**        | false                | false              |
false                     | false                | not front_loaded      | not navigable
(0)     | **selector_rich** (id=1, density=0.08) | not prose_heavy (sub=0, wavg=2.00) |
**none** (other=1) | **none** |
| V4 | v4-blog          | false           | false                | **true**           |
**true** (sem=11)         | **true** (total=1)   | **front_loaded**      | **navigable**
(5)     | not selector_rich (hooked=0, density=0.00) | **prose_heavy** (sub=5,
wavg=5.02) | **paragraph** (para=7) | **contextual** (chrome=3, aside=2) |
| V5 | v5-divsoup       | false           | false                | false              |
false                     | false                | not front_loaded      |
**fragment_only** (5) | **selector_rich** (classed=28, density=0.61) | not prose_heavy
(sub=1, wavg=5.92) | **presentational** (pres=12) | **fragment_only**
(non_chrome_frag=5) |
| H1 | h1-localbiz      | false           | **true**             | **true**           |
**true** (sem=8)          | **true** (addr=1)    | **front_loaded**      | **navigable**
(contact=2) | not selector_rich (hooked=0, density=0.00) | not prose_heavy (sub=2,
wavg=4.79) | **mixed** (para=3, list=3) | **chrome_only** (chrome=3) |
| H2 | h2-hybrid        | false           | false                | **true**           |
**true** (sem=6)          | **false**            | not front_loaded      | **navigable**
(2)     | **selector_rich** (id=4, data=3, density=0.16) | **prose_heavy** (sub=3,
wavg=4.32) | **paragraph** (para=3) | **chrome_only** (chrome=2) |
```

14. Appendix C — The v2 Prompt Template

The appendix below reproduces the repository template verbatim. Original wording is preserved for archival fidelity.

AXO Experiment — Parameter 4 Prompt Template

Reusable template for each P4 iteration. Replace placeholders in `{curly braces}` before sending. Designed to converge faster than P3 by forcing every non-NULL move to cite a concrete consumer failure.

Differences from the P3 template

- **Positive criteria change.** P3 required a new signal that varies across fixtures. P4 requires a schema-reshape move that fixes a reproducible consumer failure on the existing output.
- **Mandatory rule shape changes.** P3 required a fixture-diversity table (new field value across fixtures). P4 requires a **consumer-failure pair**: one fixture URL, one standard consumer access pattern (Python `json.loads` + typed dict access, or similar), the failing call's output BEFORE the move, and the succeeding call's output AFTER. No table of fixture values — the move is evaluated on output SHAPE, not VALUES.
- **No new signals allowed.** P3 closed at iter 14 with 11 signals. Additive moves are out of scope.
- **No value changes allowed.** A P4 move preserves every signal's value on every fixture, modulo the representation change it explicitly introduces (e.g., precision normalization).
- **Tighter cap.** P4 caps at 10 iterations with no mid-loop review.

```
AXO Parametric Divergence Mapping Experiment — Parameter 4, Iteration {N}

You are {MODEL} (Model {A/B}), running on {odd/even} iterations. Parameter 4: Output
Clarity.

Parameter 4 definition:
The script returns a structured JSON object that a scoring pipeline can consume directly
without transformation. Field names are unambiguous. Value types are consistent across
fixtures and code paths. Null handling is explicit. The key set is identical on every
code path. Numeric precision is uniform per dimension class.

Pre-registered scope (binding): experiments/axo-derivation/P4_SCOPE.md

For Parameter 4, a qualifying move fixes a CONCRETE CONSUMER FAILURE on the current
output and satisfies ALL of:
1. Reproducible consumer failure — name a consumer access pattern (e.g., Python
`json.loads(output)["foo"]["bar"]`, or `TypedDict` validation) that fails on the current
output and succeeds after the move, citing a specific fixture URL.
2. Value-preserving — every field's value on every fixture is unchanged, except for the
representation change the move explicitly introduces (e.g., precision normalization).
3. Cross-path coverage — the fix applies to every code path that emits the affected
field (success path + all error paths).
4. Non-bikeshed — renames cite a specific misleading or ambiguous aspect of the current
name. "I prefer X" is not a qualifying reason.
5. Idempotent — running the move twice is equivalent to running it once.

Qualifying move categories (exactly one per iteration):
A. Schema-consistency fix — a field is present on one path, absent on another. Make the
key set identical across all paths.
B. Type normalization — a field has different runtime types across fixtures. Pick one
type and apply uniformly.
C. Null-handling explicit — a field is `null` sometimes and missing sometimes for the
same logical reason. Make it always present (with `null` on absence) OR always absent.
```

D. Precision normalization — mixed rounding rules on float fields. Pick one precision policy per dimension class.
E. Field rename — only if the current name is materially misleading AND the replacement is unambiguous AND no in-tree consumer uses the current name.

The following do NOT count:
- Adding new measurement fields (P3 is closed at iter 14)
- Changing what a signal measures (that is a P3 correction, not P4)
- Reordering keys within a single code path (JSON is order-independent)
- Cosmetic changes: whitespace, comments, variable renames, import order, code-style refactors
- Speculative defensive checks with no reproducible consumer failure
- Bikeshed renames: personal style preferences, camelCase/snake_case debates, singular/plural debates, "I think X reads better"
- Adding fields like `schema_version`, `success: true`, `api_version` for "clarity" (additive = P3)
- Anything in P4_SCOPE.md EXCLUDE list

NULL condition:
Return NULL if you cannot propose a move that satisfies all five criteria. NULL is the correct answer when all known consumer failures on this script have been addressed, or remaining candidates are bikeshed-level, cosmetic, or require P3-style changes. Do not keep the loop alive with marginal renames.

MANDATORY CONSUMER-FAILURE PAIR (for non-NULL):

You may return non-NULL only if you can fill in this block:

```
Category: {A | B | C | D | E}
Fixture URL: http://127.0.0.1:8799/{visible/xxx.html or a reproducible off-fixture input with realistic HTTP response}
Consumer access (BEFORE the move, current iteration-{N-1}.js output):
  <code snippet, e.g., `json.loads(out)["heading_hierarchy"]` raises KeyError on the fetch-error path>
Consumer access (AFTER the move, iteration-{N}.js output):
  <same code snippet, now succeeds and returns the expected null/shape>
Value check:
  <list of field values that MUST remain identical across both outputs — at minimum, every unchanged field on every fixture>
```

If you cannot produce this block, the move does not qualify. Return NULL.

If you cannot identify a qualifying move, return exactly:

NULL — no qualifying P4 move identified for this iteration.

Then explain:
1. which moves you considered
2. why each candidate failed (which criterion — no reproducible consumer failure, value-changing, cross-path incomplete, bikeshed, out-of-scope)
3. why no remaining candidate qualifies under P4_SCOPE.md

Current script: {path-to-current-script}
Current changelog: {path-to-current-changelog}

Condensed history:
P1 locked DOM depth measurement (35 iters, no convergence). P2 locked fetch/transport handling (converged iter 18). P3 produced 11 coverage signals and converged iter 14 (Claude iter-13 NULL + Codex iter-14 NULL). P4 is the last parameter before `axo-

```
measurement-script.js` is finalized.

Moves already applied in P4 (do not re-propose):
{list of P4 moves applied so far, with before/after summary — maintained in experiment-
log.md}

Current output shape (all 17 keys, success path on a successful fixture):
```
{ url, maximum_depth, average_depth, element_count, js_dependency, structured_data,
heading_hierarchy, semantic_html, machine_readable_blocks, answer_first, link_profile,
selector_affordance, content_depth, text_container_profile, reference_context,
timestamp, error }
```

Error-path output shapes (audit these for consistency — a mismatch here is a qualifying
P4 move):
{summary of any error-path key-set drift, if any remains open}

Output requirements:
- If NULL: copy the current script unchanged to {next-script-path}
- If non-NULL: make exactly one change — one move from categories A-E. Save the result
to {next-script-path}.
- Write {next-changelog-path}

Changelog requirements:
- Category (A-E)
- Affected field(s) and code paths
- Consumer-failure pair (BEFORE / AFTER access, with specific fixture URL)
- Value-preservation verification (list of fields whose values you confirmed unchanged)
- Cross-path coverage verification (list of code paths touched)
- What you considered and rejected, with criterion cited for each rejection

Important:
Do not keep the loop alive by proposing cosmetic renames, bikeshed style changes, or
speculative defensive null-checks. The P4 space is small (schema-consistency fixes, type
normalizations, null-handling, precision, a handful of genuinely-misleading names). If
the remaining ideas fail a criterion in P4_SCOPE.md, return NULL.
```

15. Appendix D — Complete List of Accepted P3 Signals and Their Categories

Iteration	Model	Signal	Output group	Concrete agent-task category	Acceptance / redundancy note
P3-1	Claude	`js_dependency`	rendering model	no-browser routing between SSR and SPA shells	threshold case H2 prevented script-count-only overfit
P3-2	GPT/Codex	`structured_data`	knowledge graph availability	fact extraction for product or business identity	holdout H1 flipped positive, showing real generalization
P3-3	Claude	`heading_hierarchy`	document outline integrity	outline-based extraction and summarization	clean non-redundancy case versus div-soup
P3-4	GPT/Codex	`semantic_html`	semantic role richness	main-content isolation and chrome suppression	weak boolean redundancy on `present`; sub-fields rescue
P3-5	Claude	`machine_readable_blocks`	in-content machine-readable structure	tables, dates, addresses, definition lists	weak boolean redundancy on `present`; holdout H2 rescues
P3-6	GPT/Codex	`answer_first`	information front-loading	fast answer routing	accepted with explicit brittleness warning about regex leakage
P3-7	Claude	`link_profile`	link navigability profile	crawl-out decisioning and fake-nav detection	boolean `navigable` is familiar, but `fragment_only` adds a second axis
P3-8	GPT/Codex	`selector_affordance`	selector-hook richness	selector-first automation versus semantic fallback	weak boolean redundancy on visibles; both holdouts break it
P3-9	Claude	`content_depth`	prose density and depth	readability / summarization routing	weak visible redundancy rescued by both holdouts
P3-10	GPT/Codex	`text_container_profile`	text-carrier modality	choose paragraph vs table vs presentational extraction	categorical novelty survives despite one redundant projection
P3-12	GPT/Codex	`reference_context`	link location context	multi-hop research crawl decisioning	accepted for sub-field decomposition despite strong reducibility concern on `mode`

16. Appendix E — Selected Full Changelogs

The appendix below reproduces selected changelogs verbatim. Original wording is preserved for archival fidelity.

E.1 P2-18 Convergence

```
# Iteration 18 — GPT — Parameter 2 (Edge Case Handling)

## Result

**NULL.** No structural change.

Iteration 18 is identical to iteration 17. I did not identify a remaining
reproducible input that causes either:

- a crash (throw, uncaught error, non-clean termination on a valid-input edge case), or
- a fabricated measurement (numeric measurement on input that is not a measurable HTML
document under the experiment's settled P2 policy)

## What I Considered

1. **Doctype shell variants that still return the 3-element measurement**
   - `<!doctype html>`
   - `<!doctype html><!--c-->`
   - `<!doctype html>` followed by other non-element tokens

   Rejected. These are not red-state fabrications under the current experiment.
   The doctype exemption from iteration 16 is now an explicit policy choice:
   browser-equivalent empty-document shells measure as the implicit
   `HTML/HEAD/BODY` tree. Reversing that would contest the settled policy, not
   close a new crash or fabricated-measurement path.

2. **Empty HTML shells with real parsed document elements**
   - `<html></html>`
   - `<html><head></head></html>`

   Rejected. These are not fabrications. The parsed tree contains real
   `HTML`/`HEAD` elements, and the synthetic `BODY` is the browser-equivalent
   document-synthesis behavior established in Parameter 1. The resulting
   3-element measurement is within policy.

3. **Real-element non-HTML vocabularies under claimed HTML**
   - `<?xml version="1.0"?><foo><bar/></foo>`
   - `<svg><g></g></svg>`

   Rejected. These inputs do not fabricate anything. The parser produces real
   element nodes, and the script measures those nodes. Whether that is
   semantically desirable is not a Parameter 2 crash/fabrication issue.

4. **Garbage before or around a doctype**
   - `< <!doctype html>`

   Rejected. This is a speculative defensive-check candidate, not a reproduced
   realistic red state. Under the current script it falls into the doctype
   exemption and measures as 3 elements, but there is no grounded fixture
   argument that this should remain in Parameter 2 rather than being treated as
   adversarial junk outside the experiment's settled handling.

5. **BOM-prefixed HTML**
   - `\uFEFF<html><body><p>x</p></body></html>`

   Rejected. This does not fail under the current script. It already measures
   successfully, so it is not a red-state candidate.
```

```
6. **Redirect loops, decode failures, malformed URLs, and other transport errors**

   Rejected. These already resolve to clean error JSON. Adding more-specific
   branches would only change wording, which the v2 rules explicitly exclude.

## Why Each Candidate Failed The Fixture Rule

- **No reproducible crash:** none of the tested cases threw, exited uncaught, or
  terminated outside the existing clean-error path.
- **No reproducible fabrication:** the only numeric outputs I found were either
  browser-equivalent empty-document shells (`<!doctype ...>`, `<html></html>`)
  or measurements over real parsed element nodes (`svg`, custom XML-like tags).
- **Speculative only:** malformed-doctype junk does not have a persuasive
  experiment-grounded claim that it is both realistic and outside the existing
  doctype policy.

## Why NULL

The dominant P2 failure class is closed from all five observed axes:

1. transport
2. auto-parse
3. encoding
4. pre-parse body shape
5. post-parse zero-element bodies

The remaining ideas are either:

- not crashes,
- not fabrications,
- already clean errors,
- or speculative defensive checks.

Under the tightened P2 definition, there is no qualifying non-NULL move left.
```

E.2 P3-14 Independent Validation

```
# P-3 Iteration 14 — GPT/Codex — NULL

**Model:** GPT/Codex (Model B, even iterations)
**Parameter:** 3 — Coverage Completeness (AXO-specific signals)
**Result:** NULL — no qualifying new signal identified for Parameter 3.
**Starting script:** `P-3/iteration-13.js`
**Output script:** `P-3/iteration-14.js` (byte-identical to iter-13)

## Summary

I re-ran the P3 qualification test against the remaining structural seams that still
looked remotely defensible after Claude's iter-13 NULL. None survived all five criteria
simultaneously.

The three candidates worth serious consideration were:

1. `section_heading_coverage`
2. `temporal_metadata`
3. `article_header_profile`

All three are measurable from raw HTML and falsifiable on the visible fixtures. All
three fail criterion 4: they do not add a genuinely new signal once checked against the
existing P3 stack.

## Considered and rejected

### 1. `section_heading_coverage` — rejected

**Proposed shape:**

```json
"section_heading_coverage": {
  "section_count": <int>,
  "headed_section_count": <int>,
  "mode": <"complete"|"partial"|"none">
}
```

Visible-fixture measurements:

Fixture	section_count	headed_section_count	mode
V1 — v1-product	3	3	`complete`
V2 — v2-news	3	3	`complete`
V3 — v3-spa	0	0	`none`
V4 — v4-blog	4	3	`partial`
V5 — v5-divsoup	0	0	`none`

This is structurally real and task-relevant for section-by-section extraction.

Why it fails criterion 4:

On the visible fixtures, the categorical pattern is already recoverable from the current
signal stack:

- `complete` = semantic-rich pages without contextual reference links
- `partial` = semantic-rich page with contextual/aside references
- `none` = non-semantic pages
```

Concretely, the visible assignments are fully captured by the combination of:

- `semantic_html.present`
- `reference_context.mode`

Mapping on visibles:

Fixture	semantic_html.present	reference_context.mode	section_heading_coverage.mode
V1	T	`chrome_only`	`complete`
V2	T	`chrome_only`	`complete`
V3	F	`none`	`none`
V4	T	`contextual`	`partial`
V5	F	`fragment_only`	`none`

That is too close to a decomposition of existing fields rather than a new axis.

2. `temporal_metadata` — rejected

Proposed shape:

```json
"temporal_metadata": {
  "calendar_time_count": <int>,
  "duration_time_count": <int>,
  "unparsed_time_count": <int>,
  "kind": <"calendar"|"duration"|"none">
}
```

Visible-fixture measurements:

Fixture	time_count	calendar	duration	kind
V1 — v1-product	1	0	1	`duration`
V2 — v2-news	1	1	0	`calendar`
V3 — v3-spa	0	0	0	`none`
V4 — v4-blog	1	1	0	`calendar`
V5 — v5-divsoup	0	0	0	`none`

This is measurable and semantically meaningful. It distinguishes duration-style `<time datetime="PT40H">` from publication-date `<time datetime="2026-...">`.

Why it fails criterion 4:

Claude’s iter-13 analysis was correct. Every usable projection collapses onto partitions that are already represented:

- `duration_time_count > 0` isolates V1 only
- `calendar_time_count > 0` yields `F/T/F/T/F`
- `kind != none` yields `T/T/F/T/F`

Those patterns are already captured by combinations of:

- `machine_readable_blocks.time_count`
- `content_depth.prose_heavy`
- `text_container_profile.dominant_block_type`

```
The signal is semantically novel, but not structurally novel enough on this fixture set.

### 3. `article_header_profile` — rejected

**Proposed shape:**

```json
"article_header_profile": {
  "article_count": <int>,
  "article_header_count": <int>,
  "article_header_time_count": <int>,
  "article_header_paragraph_count": <int>
}
```

Visible-fixture measurements:

| Fixture | article_count | article_header_count | article_header_time_count |
article_header_paragraph_count |
|---------|---------------|----------------------|---------------------------|----------
----------------------|
| V1 — v1-product | 1 | 0 | 0 | 0 |
| V2 — v2-news | 1 | 1 | 1 | 1 |
| V3 — v3-spa | 0 | 0 | 0 | 0 |
| V4 — v4-blog | 1 | 1 | 1 | 1 |
| V5 — v5-divsoup | 0 | 0 | 0 | 0 |

Why it fails criterion 4:

This reproduces the same article-prose split already exposed by:

- `content_depth.prose_heavy`
- `text_container_profile.dominant_block_type == "paragraph"`

Visible pattern: `F/T/F/T/F`.

It adds no new distinction beyond “article-shaped prose page versus everything else,”
which the current schema already measures.

Why no remaining candidate qualifies

After iter 12’s `reference_context`, the remaining visible-fixture variation is mostly
one of:

1. flat axes
2. thin one-fixture isolators
3. recombinations of existing sub-fields

The plausible remaining signals now fail in one of two ways:

- they are structurally real but collapse onto already-saturated visible patterns
- they introduce semantics that the current fixtures do not exercise independently

At this point, continuing would mean keeping the loop alive with recomposed versions of
existing fields. That violates the standing rule. Under `P3_SCOPE.md`, `NULL` is the
correct answer.

Output schema change summary

None. `P-3/iteration-14.js` is byte-identical to `P-3/iteration-13.js`.
```

E.3 P4-5 Broken NULL

```
# P-4 Iteration 5 — Claude — NULL

**Model:** Claude (Model A, odd iterations)
**Parameter:** 4 — Output Clarity
**Result:** NULL — no qualifying P4 move identified for this iteration
**Starting script:** `P-4/iteration-4.js`
**Output script:** `P-4/iteration-5.js` (byte-identical to iter-4)

## Why NULL

After iter-4, the four pre-registered P4 issues are all closed:

1. Catch-block key-set drift → closed at iter-1 (category A)
2. `heading_hierarchy` accessor cliff → closed at iter-2 (category C/B)
3. Mixed precision on three float fields → closed at iter-3 (category D)
4. Nested-group object/null asymmetry → closed at iter-4 (category B)

The output schema is now:
- 17 top-level keys on every one of the 11 code paths
- All 11 nested field groups are object-always with identical leaf-key sets across
success and error paths (verified: `Object.keys(out[g])` is identical whether the
fixture succeeded or errored)
- Three float fields (`average_depth`, `content_depth.weighted_avg_depth`,
`selector_affordance.hook_density`) normalized to 2-decimal precision uniformly
- No cross-fixture type drift on success-path leaves (verified by walking all 5 visible
outputs and diffing each leaf's runtime type)

## Candidates considered and rejected

1. **Zero-fallback `0` → `0.0` normalization on the three float fields** (category B).
   - Fails criterion 1 (reproducible consumer failure) *as a P4 output-shape move*. The
ternary `elementCount > 0 ? parseFloat(...) : 0` returns integer `0` when elementCount
is zero, but `JSON.stringify(0)` and `JSON.stringify(0.0)` both emit the same two-
character string `"0"`. The output JSON bytes do not change regardless of whether the
internal JavaScript value is `0` or `0.0`. A Python consumer reading `"0"` gets `int(0)`
in either case.
   - Fixing the int-vs-float runtime-type drift would require one of: (a) custom JSON
serialization emitting `0.0` manually, (b) using a sentinel like `0.01` or `null` in
place of the zero fallback, (c) dropping the zero-fallback entirely and letting the
field be `null` when `elementCount === 0`. All three are either invasive (a), value-
changing (b), or ambiguous between "zero elements measured" and "measurement absent"
(c).
   - Additionally, the zero-fallback is likely **dead code** in practice: the "zero-
element post-parse" error guard at line 1206 returns before `measureDepth()` is called
when `hasParsedElementNodes()` is false. A quick audit of the fixture suite shows no
successful fixture reaches measureDepth with `elementCount === 0`. If the path is
unreachable, fixing it satisfies no consumer failure.
   - Deferred / effectively closed.

2. **Normalize array-typed leaves (`structured_data.schema_types`,
`semantic_html.tags_present`) to `[]` on error paths instead of `null`** (category B,
accessor-cliff prevention).
   - Fails the null-sentinel convention that P4_SCOPE.md explicitly endorses: *"Fix:
normalize to a single declared type with an explicit null sentinel for absence."*
Converting `null` → `[]` removes the distinction between "could not measure" (error
path) and "measured, found zero" (success path with empty result).
   - More importantly, a consumer doing `len(out["structured_data"]["schema_types"])` on
an error-path output would want to know the measurement failed — raising `TypeError` is
```

arguably the correct signal, not silently returning 0. The existing top-level `error` field already carries the error condition; consumers should branch on `out["error"]` once at the top before iterating leaves.
   - Rejected on convention grounds, not criterion 1.

3. **Rename `error` → `error_message` / `error_string`.**
   - Fails criterion 4 (non-bikeshed). The current `error` field is null on success and string on error; this is idiomatic for a "summary of what went wrong if anything". No reasonable consumer misreads `error: "HTTP 404"` as anything other than a message.

4. **Rename `schema_types` → `structured_data_types` for self-describing naming.**
   - Fails criterion 4. The field is nested inside `structured_data`, so its accessor is already `structured_data.schema_types` — self-qualifying. The proposed rename makes the accessor `structured_data.structured_data_types`, which is verbose without gain.

5. **Rename `hook_density` → `selector_hook_density`.**
   - Fails criterion 4. Nested under `selector_affordance`. Same argument as above.

6. **Rename `mode` (inside `reference_context`) to `link_context_mode`.**
   - Potentially less bikeshed: `mode` by itself is ambiguous outside its nested context. But the accessor `reference_context.mode` is already self-qualifying in-tree, and no external consumer currently references the flat field name.
   - Rejected as bikeshed by the same argument as candidates 4 and 5.

7. **Rename `present` (appears as a boolean leaf in multiple nested groups: `structured_data.present`, `semantic_html.present`, `machine_readable_blocks.present`) to something more specific per-group.**
   - Considered, but the flat name `present` is already specific once nested (`structured_data.present` means "any structured data present"). Renaming each instance to `structured_data_present` / `semantic_html_present` / etc. duplicates the parent group name in the leaf name. Fails criterion 4.

8. **Add a top-level `status: "success" | "error"` field for clearer error-vs-success branching.**
   - Rejected as additive (P3 territory, closed). P4_SCOPE.md EXCLUDE list: "Adding new output fields for 'clarity' (e.g., a `schema_version` key, a `success: true` boolean). Additive changes are P3 territory."

9. **Key ordering consistency across paths.**
   - JSON is order-independent; reordering keys is cosmetic. Explicitly listed in P4_SCOPE.md EXCLUDE.

10. **Leaf-level type drift across fixtures.**
    - Empirically checked: walked all 5 visible fixture outputs (v1-v5) with iter-4's script, recorded the runtime type of every leaf on every fixture, and diffed. Zero drift — every leaf has the same type across all 5 visibles on the success path. No remaining category-B candidate.

Status

- **First Claude NULL of P4.**
- Iter 6 (Codex) determines whether bilateral convergence is reached.
- If Codex iter-6 also NULLs → bilateral convergence at iter-6, P4 closes at 6/10 (4 under the cap).
- If Codex iter-6 finds a concrete consumer failure I missed → proceed.

The P4 space was correctly pre-registered as narrow: P4_SCOPE.md called out catch-block drift, mixed precision, and nested-group shape as the primary known-open issues, and all three are now closed. Iter-4's bulk normalization was the last structural move in the registered space. Remaining candidates are invasive (zero-fallback), convention-violating (array leaves), or bikeshed (renames).

```
## Holdout predictions

iter-5 is byte-identical to iter-4. Both holdouts produce output identical to iter-4 on
both holdouts. No behavior change.
```

E.4 P4-7 Broken NULL

```
# P-4 Iteration 7 — Claude — NULL

**Model:** Claude (Model A, odd iterations)
**Parameter:** 4 — Output Clarity
**Result:** NULL — no qualifying P4 move identified
**Starting script:** `P-4/iteration-6.js`
**Output script:** `P-4/iteration-7.js` (byte-identical to iter-6)

## Correcting iter-5's methodology

Iter-5 NULL was broken by Codex iter-6 because I checked cross-fixture type drift using
JavaScript `typeof`, which returns `'number'` for both `2` and `2.1`. Python's
`json.loads` distinguishes `2` (int) from `2.00` (float). For iter-7 I switched to
**byte-level JSON-literal inspection** via regex on the raw stdout bytes, which is what
Python consumers actually see.

## Empirical drift hunt (iter-7)

Four checks run against `P-4/iteration-6.js` on all 5 visible fixtures + the catch-
branch error path:

1. **Top-level key order** — identical across all 6 outputs (5 visibles + 1 error):
`url, maximum_depth, average_depth, element_count, js_dependency, structured_data,
heading_hierarchy, semantic_html, machine_readable_blocks, answer_first, link_profile,
selector_affordance, content_depth, text_container_profile, reference_context,
timestamp, error`.

2. **Nested-group key order** — all 11 groups have identical leaf-key order across all 6
outputs. No cross-path drift.

3. **JSON-byte int/float drift** — for every numeric leaf, regex-extracted the JSON
literal per fixture and compared whether a decimal point is present. No drift found on
iter-6. (The three designated float fields all emit as `X.XX` now; the integer counts
all emit without a decimal point consistently.)

4. **Array/string/null leaf types on success path** — `schema_types` is always array
(can be `[]` as on V2/V3/V4/V5), `tags_present` is always array (can be `[]` as on
V3/V5), `dominant_block_type` is always string (including literal `"none"` on V3),
`reference_context.mode` is always string (including `"none"` on V3). No cross-fixture
type drift at JSON-byte level.

Result: no qualifying consumer-failure evidence surfaces.

## Candidates considered and rejected

1. **Any remaining JSON-byte int/float drift** — none found via regex scan across 5
visibles.

2. **Cross-path key-order divergence** — none found; all 11 code paths (success and
every error branch examined) emit keys in identical order.

3. **Leaf-level type drift between error-path (null) and success-path (typed)** — this
is the intended null-sentinel pattern P4_SCOPE.md endorses. Converting null leaves to
typed defaults (`[]` for arrays, `0` for counts, `false` for booleans, `""` for strings)
would erase the distinction between "could not measure" and "measured, and the measured
result is zero/empty/false". Consumer branching on the top-level `error` field already
handles the error case cleanly.
```

4. **Sentinel-replace serializer fragility** (Codex iter-6 implementation) — theoretical collateral if a string field matched `/__AXO_FLOAT__.*__/`, but `error` messages are HTTP-status-style strings and `url` is user-provided. Current fixture suite has no collision. A cleaner implementation would use field-name-anchored regex instead of sentinels, but that is refactoring not a consumer-failure fix. Not a qualifying P4 move.

5. **Category E renames** — re-examined the same rename candidates from iter-5 (`error`, `schema_types`, `hook_density`, `mode`, `present`, `average_depth`) plus `total` (ambiguous inside `machine_readable_blocks.total`) and `valid` (inside `heading_hierarchy.valid`). Every nested field has a self-qualifying accessor path (`structured_data.present`, `machine_readable_blocks.total`, `heading_hierarchy.valid`). No rename clears the "materially misleading" bar from P4_SCOPE.md criterion 4.

6. **Precision on integer fields** — all integer counts emit without trailing zeros (e.g., `element_count: 2`, not `element_count: 2.0`). This is correct — they are ordinal counts, not ratios. Converting them to `2.0` would be a value-type change, not a precision normalization.

7. **`"none"` string sentinel on `dominant_block_type` / `reference_context.mode`** — flagged but intentional. V3 emits `dominant_block_type: "none"` (measured, no dominant type) vs an error path emits `null` (could not measure). The distinction is meaningful; collapsing both to `null` or both to `"none"` would lose information. Not a P4 move.

8. **Adding `status: "success" | "error"` top-level field** — additive (excluded by P4_SCOPE.md EXCLUDE list).

9. **Key reordering within a path** — cosmetic, excluded by P4_SCOPE.md.

10. **Zero-fallback `0` in the `measureDepth` ternary** — Codex iter-6's serializer forces all three float fields to `X.XX` format regardless, so even the theoretically-reachable-but-likely-dead zero-fallback emits as `0.00` on output. Closed.

Why this NULL is defensible (unlike iter-5)

Iter-5 used the wrong measurement tool (JS `typeof`). Iter-7 uses the consumer's measurement tool (JSON byte-level regex) and finds no drift. This is the correct audit surface for P4 — the same surface Codex used to find the iter-6 move.

Remaining candidates are either:
- rejected by convention (null-sentinel for absence, "none" as measurement result),
- additive (P3 territory, closed),
- cosmetic (key reordering inside one path),
- or bikeshed (renames with self-qualifying accessor paths).

P4 is substantively converged after iter-6. The open set is empty of concrete consumer failures on the fixture set.

Holdout predictions

iter-7 is byte-identical to iter-6. Both holdouts produce the same output as iter-6 did on them. No behavior change.

Status

- Second Claude NULL attempt in P4 (first was iter-5 — correctly challenged by Codex).
- If Codex iter-8 NULLs → bilateral convergence at iter-8, P4 closes at 8/10 (2 under cap).
- If Codex iter-8 finds another concrete consumer failure I missed → proceed.

Invitation to Codex

```
If Codex iter-8 non-NULLs, I would specifically look for:
- any JSON-byte-level drift I missed on the success path (beyond int/float, which iter-6
closed),
- any key-set mismatch between the `createNullSignalGroups()` template and the actual
success-path emission for a specific group (though I spot-checked and parity holds),
- any overlooked field type that varies across fixtures in a way Python/TypeScript
consumers would branch on.
```

E.5 P4-8 Adversarial-Input Find

```
# P-4 Iteration 8 — GPT/Codex — serializer collision fix

**Model:** GPT/Codex (Model B, even iterations)
**Parameter:** 4 — Output Clarity
**Category:** B (Type normalization)
**Result:** NON-NULL — removed a real output-type corruption in the iter-6 float
serializer
**Starting script:** `P-4/iteration-7.js`
**Output script:** `P-4/iteration-8.js`

## What changed

Replaced iter-6's global sentinel-replace serializer with key-anchored numeric rewrites.

Before iter-8, `stringifyOutput()`:

1. rewrote the three designated float fields to sentinel strings like
`__AXO_FLOAT__2.00__`
2. ran `JSON.stringify(payload)`
3. globally regex-replaced any JSON string whose full contents matched the sentinel
pattern

That worked for the intended float fields, but it also corrupted unrelated string fields
if their value happened to equal the sentinel pattern.

After iter-8, `stringifyOutput()`:

- starts from plain `JSON.stringify(output)`
- rewrites only these exact JSON keys:
  - `average_depth`
  - `hook_density`
  - `weighted_avg_depth`

No other string or numeric field is touched.

## Why this qualifies

This is a real category-B consumer failure:

- the top-level `url` field is supposed to be a string on every path
- on iter-7, a malformed but reproducible input like `__AXO_FLOAT__2.00__` caused the
emitted JSON to contain `"url":2.00`
- a downstream consumer parsing the output saw `float` instead of `str`

That is not theoretical fragility. It is a live type corruption on a reproducible input.

## Consumer-failure pair

```text
Category: B
Fixture URL: off-fixture reproducible input `__AXO_FLOAT__2.00__`

Consumer access (BEFORE the move, P-4/iteration-7.js output):
  out = json.loads(stdout)
  type(out["url"]).__name__, out["url"]
  # ("float", 2.0)
  # raw JSON begins with: {"url":2.00,...,"error":"Invalid URL"}
```
```

```
Consumer access (AFTER the move, P-4/iteration-8.js output):
  out = json.loads(stdout)
  type(out["url"]).__name__, out["url"]
  # ("str", "__AXO_FLOAT__2.00__")
  # raw JSON begins with: {"url":"__AXO_FLOAT__2.00__",...,"error":"Invalid URL"}

Value check:
  Only the corrupted `url` token changes on the malformed-input path.
  The three intended float-designated fields keep the iter-6 formatting rule.
```

Second proof of the same bug class:

```text
Input: __AXO_FLOAT__-7.50__
BEFORE (iter-7): type(out["url"]).__name__ == "float", value == -7.5
AFTER  (iter-8): type(out["url"]).__name__ == "str",   value == "__AXO_FLOAT__-7.50__"
```

Red-state / green-state output quote

Red state (`iteration-7.js`):

```json
{"url":2.00, ... "error":"Invalid URL"}
```

Green state (`iteration-8.js`):

```json
{"url":"__AXO_FLOAT__2.00__", ... "error":"Invalid URL"}
```

Value-preservation verification

Verified the intended float-formatting behavior still holds on visible success fixtures:

- V3 `content_depth.weighted_avg_depth` still emits `2.00`
- V4 `selector_affordance.hook_density` still emits `0.00`
- non-whole-number float tokens remain unchanged (`1.69`, `3.97`, `4.81`, `5.02`, `0.08`, `0.61`, etc.)

This is therefore a pure narrowing of the serializer's blast radius:

- **before:** any full-string sentinel-shaped JSON string could be rewritten
- **after:** only the three intended numeric keys are rewritten

No measurement values changed.

Cross-path coverage

The fix applies to every emitted payload because every code path still goes through `printOutput()` / `stringifyOutput()`:

1. no-URL guard
2. HTTP 204/205
3. HTTP 206
4. non-HTML content type
5. unsupported charset
6. auto-parsed non-HTML body
7. empty body

```
8. body does not start with a tag
9. zero-element post-parse
10. success path
11. top-level `catch (err)`

The three target float fields remain normalized wherever they are numeric.
All unrelated strings now remain strings.

## Verification

- `node --check P-4/iteration-8.js` passed
- `node P-4/iteration-7.js __AXO_FLOAT__2.00__` emitted `"url":2.00`
- `node P-4/iteration-8.js __AXO_FLOAT__2.00__` emitted `"url":"__AXO_FLOAT__2.00__"`
- Python proof:
  - iter-7: `float 2.0`
  - iter-8: `str __AXO_FLOAT__2.00__`
- Visible-fixture control runs still show:
  - V3 `weighted_avg_depth: 2.00`
  - V4 `hook_density: 0.00`

## Considered and rejected

1. **NULL**
   - Rejected because iter-7's serializer collision is an actual reproducible output-
shape failure, not a hypothetical cleanup.

2. **Leave iter-7 as-is because the collision is "unlikely"**
   - Rejected. P4 is about direct pipeline safety, and the bug is trivially reproducible
on the malformed-URL path.

3. **Remove fixed-width float formatting entirely**
   - Rejected. That would reopen iter-6's genuine Python int/float drift on visible
fixtures.

4. **Broader refactor of output printing**
   - Rejected. Not needed. The key-anchored regex replacement fixes the concrete failure
with smaller surface area than the sentinel approach.
```

E.6 P4-9 Defensible NULL

```
# P-4 Iteration 9 — Claude — NULL

**Model:** Claude (Model A, odd iterations)
**Parameter:** 4 — Output Clarity
**Result:** NULL — no qualifying P4 move identified
**Starting script:** `P-4/iteration-8.js`
**Output script:** `P-4/iteration-9.js` (byte-identical to iter-8)

## Why this NULL differs from iter-5 and iter-7

This is the **third Claude NULL attempt in P4**. The first two were broken:

- **iter-5** — used JS `typeof` to check cross-fixture type drift. Missed the Python
int/float distinction (`JSON.stringify(2)` vs `JSON.stringify(2.00)` are different
bytes, different Python types). Codex iter-6 correctly broke it with the fixed-width
float serializer.
- **iter-7** — corrected the methodology to byte-level JSON-literal regex, but
**explicitly flagged iter-6's sentinel-replace serializer fragility and dismissed it as
"refactoring not a consumer-failure fix"**. That dismissal was wrong. Codex iter-8
demonstrated a reproducible consumer failure: `node iteration-7.js
'__AXO_FLOAT__2.00__'` caused the URL-echo in the output to corrupt from string to
float. Adversarial/pathological inputs DO count for P4 consumer-failure pairs.

iter-9 takes those two lessons and applies the corrected audit surfaces:

1. **Byte-level JSON-literal inspection** (iter-7's correction) on all fixture paths
2. **Adversarial-input probing of the serializer** (iter-8's correction) — specifically
testing whether the key-anchored regex introduced at iter-8 can still be triggered into
rewriting bytes inside unrelated string values

Both surfaces are now negative.

## Audit surfaces checked

### 1. Byte-identical output between iter-6 and iter-8 on all 7 fixtures

Confirmed: all 5 visible fixtures (v1-v5) and both holdouts (h1, h2) produce identical
output between iter-6 and iter-8, modulo timestamp. iter-8 is pure serializer-scope
narrowing; it changes nothing that was already correct.

### 2. Cross-path leaf-key parity

Verified across success, DNS-failure catch-branch, and invalid-URL catch-branch: all 17
top-level keys present, all 11 nested groups present with identical leaf-key sets. Zero
drift.

### 3. Cross-fixture leaf type drift (Python level)

Walked all 7 fixtures through iter-8, recorded Python `type(v).__name__` for every leaf,
diffed. **Zero type drift** — every leaf has the same Python type across all 7 fixtures.

### 4. Cross-fixture JSON-byte int/float drift

Regex-scanned the raw stdout of all 7 fixtures for `"key":numeric-literal` tokens,
recorded whether each numeric had a decimal point. **Zero drift at the byte level** — no
numeric field emits as int on one fixture and float on another. The three designated
float fields (`average_depth`, `hook_density`, `weighted_avg_depth`) all emit as `X.XX`;
all count fields emit as bare ints.
```

5. Serializer injection robustness (iter-8 regex)

The iter-8 serializer uses three key-anchored regex replacements:

```js
.replace(/("average_depth":)(-?\d+(?:\.\d+)?)([,}])/g, ...)
.replace(/("hook_density":)(-?\d+(?:\.\d+)?)([,}])/g, ...)
.replace(/("weighted_avg_depth":)(-?\d+(?:\.\d+)?)([,}])/g, ...)
```

For the regex to match inside a string value (not just at real field positions), the byte sequence `"average_depth":N,` would need to appear in the serialized output without any intervening `\` characters. But JSON.stringify always escapes `"` inside string values as `\"`, and the key portion itself begins AND ends with `"`. Any embedded attempt produces bytes like `\"average_depth\":N,` — the `\` before the closing `"` of the key name breaks the regex anchor.

Tested empirically with six adversarial inputs:

input	url field in output	corruption?
`xx","average_depth":5,"yy`	string `xx","average_depth":5,"yy`	no
`xxx","average_depth":999}`	string `xxx","average_depth":999}`	no
`xxx","hook_density":0.5,x`	string `xxx","hook_density":0.5,x`	no
`"average_depth":42`	string `"average_depth":42`	no
`http://example.com/  `	string `http://example.com/  `	no
`http://example.com/<5KB-of-a's>`	string (full, unmodified)	no

All six pass through as strings with their original content intact. The iter-8 key-anchored regex is genuinely robust against string-value injection because JSON escaping breaks the anchor pattern.

6. Error-message content stability

`err.message` flows into the `error` field as a string. Control characters (newlines, tabs) are JSON-escaped. Number-like substrings inside error messages can't trigger the serializer regex because they're inside escaped string values (same analysis as #5).

7. Timestamp format stability

`new Date().toISOString()` always emits `YYYY-MM-DDTHH:MM:SS.sssZ` (24 chars, UTC). Stable across all 11 paths.

8. Number edge cases

- `NaN`, `Infinity` — `JSON.stringify` emits as `null`; regex doesn't match null, stays null.
- Negative zero — `JSON.stringify(-0)` emits `0`; serializer formats to `0.00`. Fine.
- Integer overflow (> 2^53) — not reachable on current fixtures (counts bounded by DOM size).
- Idempotence — `Number("2.00").toFixed(2) === "2.00"`; second pass is no-op.

Candidates considered and rejected

1. **Any residual byte-level int/float drift** — none found across 7 fixtures (check #4).
2. **Any residual cross-path key drift** — none found across success + 2 catch variants (check #2).
3. **Any residual cross-fixture leaf type drift** — none found (check #3).
4. **iter-8 serializer injection surface** — safe by JSON-escaping construction,

verified empirically on 6 adversarial inputs (check #5). No remaining refactor target that would qualify as a consumer failure.
5. **`url: null` on no-arg / empty-string paths vs `url: string` on all other paths** — this is the null-sentinel convention for absence that P4_SCOPE endorses. `null` signals "no URL was provided"; the `error` field already carries the error condition. Converting `null` → `""` (empty string) would erase the absence distinction.
6. **`"none"` vs `null` on `dominant_block_type` / `reference_context.mode`** — intentional distinction from iter-7 analysis: `"none"` is a measurement result ("measured, no dominant type") vs `null` is absence ("could not measure"). Collapsing either way loses information.
7. **Array leaves (`schema_types`, `tags_present`) emitting `null` on error paths vs `[]` on success-with-empty** — same null-sentinel rationale. `null` signals absence; `[]` signals measured-empty. P4_SCOPE endorses this distinction.
8. **Renames** — re-examined for the third time: `error`, `schema_types`, `hook_density`, `mode`, `present`, `average_depth`, `total`, `valid`, `url`. Every field is self-qualifying via its accessor path once nested. No rename clears the "materially misleading" bar from P4_SCOPE criterion 4.
9. **Adding `status: "success" | "error"` field** — additive, excluded by P4_SCOPE EXCLUDE list (P3 territory, closed).
10. **Key reordering within a path** — cosmetic, excluded by P4_SCOPE.
11. **Replacing key-anchored regex with `JSON.stringify` replacer function** — cleaner implementation (no post-hoc regex), but not a consumer-failure fix. The current implementation is robust against all adversarial inputs I could construct. Refactoring for aesthetics fails criterion 1.
12. **Precision on integer fields (counts → floats)** — counts are ordinal, not ratios. Emitting `element_count: 2.0` would be a value-type change, not a precision normalization. No consumer failure.
13. **Zero-fallback `0` dead-code** — iter-8 serializer formats `0` → `0.00` if reached; path is guarded unreachable by the zero-element-post-parse check at line 1224; closed regardless.

Holdout predictions

iter-9 is byte-identical to iter-8. Both holdouts produce output identical to iter-8 on them. No behavior change.

Status

- **Third Claude NULL attempt in P4** (iter-5 broken by iter-6; iter-7 broken by iter-8).
- Burden of proof explicitly addressed: **both prior NULL failure modes are now closed** — methodology (JS typeof → byte-level regex) at iter-7, scope (fixture-only → adversarial inputs) at iter-9.
- If Codex iter-10 NULLs → bilateral convergence at iter-10, hitting the hard cap.
- If Codex iter-10 finds a concrete consumer failure I missed → this is the last iteration of P4 regardless (hard cap 10).

Invitation to Codex

If iter-10 non-NULLs, specifically look for:
- any output-shape surface I did NOT probe (e.g., behavior under piped/non-terminal stdout, behavior with very specific HTTP error shapes like HTTP 3xx redirects that axios auto-follows vs explicit limits),
- any schema-coherence issue introduced by the iter-4 `createNullSignalGroups()` helper's specific key ordering that differs subtly from the success path,
- any edge case in the iter-8 key-anchored regex I missed (e.g., does it handle a field value of exactly `-0` correctly? What about a JSON number in exponential notation, though `toFixed` shouldn't produce one?).

If iter-10 NULLs, P4 converges at the hard cap with 3 Claude NULLs (2 broken, 1

```
defensible) and 4 Codex non-NULLs (all broke the prior NULL or extended a prior find).
That asymmetry is itself a P4 finding worth recording in the paper.
```

E.7 P4-10 Convergence

```
# P-4 Iteration 10 — GPT/Codex — NULL

**Model:** GPT/Codex (Model B, even iterations)
**Parameter:** 4 — Output Clarity
**Result:** NULL — no qualifying P4 move identified for this iteration
**Starting script:** `P-4/iteration-9.js`
**Output script:** `P-4/iteration-10.js` (byte-identical to iter-9)

## Why NULL

After re-auditing the remaining P4 surface on `iteration-9.js`, I do not see a
qualifying move that satisfies all five P4 criteria:

1. reproducible consumer failure
2. value-preserving except for the explicit representation change
3. cross-path coverage
4. non-bikeshed
5. idempotent

The four original registered issues are closed, and the two post-hoc issues I found are
also closed:

- catch-path key drift — closed at iter-1
- nested `heading_hierarchy` accessor cliff — closed at iter-2
- mixed float precision — closed at iter-3
- nested-group object/null asymmetry — closed at iter-4
- JSON-consumer int/float drift on whole-number float fields — closed at iter-6
- serializer collision corrupting `url` on sentinel-shaped input — closed at iter-8

`iteration-9.js` is already the post-iter-8 key-anchored serializer version. The
remaining space is either convention-preserving null sentinels, bikeshed renames, or
refactors without a live consumer failure.

## What I considered and why it failed

1. **Residual serializer corruption**
   - Failed criterion 1. Claude already probed the meaningful adversarial class that
broke iter-7: inputs that try to smuggle `average_depth`, `hook_density`, or
`weighted_avg_depth` into string values. With the key-anchored regex, those stay strings
because JSON escaping prevents the regex from matching inside quoted content.
   - I did not find a new reproducible input that changes any unrelated field's type or
value.

2. **`url` field type normalization (`null` vs string)**
   - Failed criterion 4 and the P4 scope convention. `url: null` on the no-arg path is
the explicit null sentinel for absence. Replacing it with `""` or some other string
would erase that distinction rather than clarify it.

3. **Array leaves on error paths (`schema_types`, `tags_present`)**
   - Failed criterion 2 / scope convention. Converting `null` to `[]` would collapse
"could not measure" into "measured and found zero". P4_SCOPE explicitly endorses the
null sentinel for absence.

4. **`"none"` vs `null` on `dominant_block_type` / `reference_context.mode`**
   - Failed criterion 2. `"none"` is a measured result; `null` is absence. Collapsing
them loses information.

5. **Rename candidates**
```

```
   - Failed criterion 4. `error`, `schema_types`, `hook_density`, `mode`, `present`,
`total`, `valid`, and `average_depth` are all self-qualifying via their accessor paths.
None is materially misleading enough to justify a rename.

6. **Additional numeric normalization**
   - Failed criterion 1. The only real numeric clarity bug was the Python int/float
split on the three float-designated fields, and iter-6/8 closed it. Integer counts
correctly remain integers.

7. **Output-printing refactor**
   - Failed criterion 1. A cleaner serializer implementation may exist, but without a
concrete consumer failure on the current output, that is refactoring, not a qualifying
P4 move.

## Why no remaining candidate qualifies

The last live non-NULL class in P4 was serializer-scope correctness. That ended at iter-
8. After that, the output is:

- schema-stable across paths
- leaf-key-stable across nested groups
- byte-stable on the visible fixture set
- Python-type-stable on the visible fixture set
- resilient to the adversarial string-injection class that broke iter-7

What remains is not ambiguity in the emitted JSON. It is preference.

That is exactly where P4 is supposed to stop.
```

---

17. Appendix F — Reproducibility Commands

F.1 Environment

- **Node.js:** 20.x LTS.
- **Python:** 3.11+ (for consumer-failure reproduction in P4).
- **Operating system:** macOS 15.x (darwin 25.4.0). Portable to Linux.
- **Hardware:** No GPU or special hardware required.

F.2 Fixture serving

Fixtures are served locally via a small HTTP server (see `fixtures/README.md` in the repository). The server binds to a configurable port, and all fixture URLs in the experiment log refer to `localhost:<port>/fixtures/<id>`.

F.3 Running a single iteration manually

```
# From the repository root.
npm install
npm run fixtures                                              # Start fixture server
node P-4/iteration-10.js http://localhost:<port>/fixtures/v1  # Run final P4 script
```

F.4 Scope documents

All scope documents are checked into the repository and frozen at parameter-start:

- `EXPERIMENT_PLAN.md` — master plan covering Part 1 (this paper) and Part 2 (Syphon integration, out of scope here).
- `PROMPT_TEMPLATE.md` — shared v2 prompt template (from P2 iteration 16 onward).
- `PROMPT_TEMPLATE_P3.md`, `PROMPT_TEMPLATE_P4.md` — parameter-specific templates.
- `P3_SCOPE.md`, `P4_SCOPE.md` — parameter-specific scope documents.
- `prompts/MASTER_CODEX_PROMPT.md` — master reusable prompt for Codex iterations (from P3 iteration 2 onward).
- `prompts/codex-iter-{2..18}.md` — per-iteration archived Codex prompts (P2 only).

F.5 Artifact versioning

Every iteration produces two files:

- `P-{n}/iteration-{m}.js` — the script at that iteration.
- `P-{n}/iteration-{m}-changelog.md` — the model's rationale for the change or NULL.

Parameter baselines are separately named:

- `p2-baseline.js` = `P-1/iteration-35.js`.
- `p3-baseline.js` = `P-2/iteration-18.js`.
- `p4-baseline.js` = `P-3/iteration-14.js`.
- `p4-final.js` = `P-4/iteration-10.js`. (This is the artifact intended for integration into AXO scoring.)

F.6 Model-version caveats

Claude and Codex were the two services used; specific model versions were those offered during April 2026 runs. No mechanism exists to replay the same model versions retrospectively. Replication attempts will necessarily use newer (or alternative) model versions and may reach different convergence points. The protocol — not the output — is what the paper claims is reproducible.

F.7 License and authorship

The repository is released under a permissive license (see `LICENSE`). The paper and experiment are authored solely by Siddharth Nigam; no co-authorship is asserted on behalf of the models whose outputs constitute the experimental data. Model outputs are treated analogously to instrument readings: recorded, cited, and analyzed, without attribution of authorship to the instrument.

References

Artstein, R., & Poesio, M. (2008). *Inter-Coder Agreement for Computational Linguistics.* Computational Linguistics, 34(4), 555–596.

Bai, Y., et al. (2022). *Constitutional AI: Harmlessness from AI Feedback.* Anthropic technical report.

Brin, S., & Page, L. (1998). *The Anatomy of a Large-Scale Hypertextual Web Search Engine.* Computer Networks and ISDN Systems.

Cutts, M. (2006). *SEO advice: writing useful articles that people will want to read and link to.* Web search blog.

Du, Y., et al. (2023). *Improving Factuality and Reasoning in Language Models through Multiagent Debate.* arXiv:2305.14325.

Liang, P., et al. (2022). *Holistic Evaluation of Language Models (HELM).* arXiv:2211.09110.

Liu, Y., et al. (2023). *G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment.* arXiv:2303.16634.

Madaan, A., et al. (2023). *Self-Refine: Iterative Refinement with Self-Feedback.* arXiv:2303.17651.

Schema.org. (2023). *Schema.org vocabulary for structured data on the Internet.* https://schema.org/

Shinn, N., et al. (2023). *Reflexion: Language Agents with Verbal Reinforcement Learning.* arXiv:2303.11366.

Srivastava, A., et al. (2022). *Beyond the Imitation Game: Quantifying and extrapolating the capabilities of language models (BIG-bench).* arXiv:2206.04615.

Wang, Y., et al. (2023). *Self-Consistency Improves Chain of Thought Reasoning in Language Models.* arXiv:2203.11171.

Zheng, L., et al. (2023). *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena.* arXiv:2306.05685.

End of paper. See `paper/NOTES.md` *for a list of claims in this paper that are not substantiated by in-repo artifacts and would require external evidence before publication.*